claude-windows / 96963dab-d38f-488c-af44-530ddbeefed1

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\96963dab-d38f-488c-af44-530ddbeefed1\subagents\workflows\wf_00bebd3c-82f\agent-a67de38c134ef59c7.jsonl`
- SHA-256: `6ee6fe7e0fda41ca31853c8875339b62e3159e35051d654fff7e7290e936b359`
- Source modified: `2026-07-08T05:52:34+00:00`
- Imported at: `2026-07-08T15:59:27+00:00`
- project: `wf_00bebd3c-82f`
- session_id: `96963dab-d38f-488c-af44-530ddbeefed1`

Transcript

1. user (2026-07-08T05:52:33.567Z)

You are a completeness critic for a design/implementation task on the repo at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer (read-only).
The task (#524): design doc resolving compute topology (Cloud Run GPU Service vs GCE L4 VM vs hybrid presigned-upload-to-L4) + two config-guarded wins: validation-on-worker placement, and trusting the upload's streamed video_id instead of re-hashing in orchestration.py.
Here is the combined subsystem map produced by 8 readers (JSON): [{"summary":"UPLOAD FLOW subsystem: the browser chunked-upload router (backend/api/uploads.py) streams sequential ?95MB parts either into a local partial file (appliance) or directly through a storage.open_writer into gs://<input_root>/uploads/<upload_id>/<filename> (hosted, #480/#471 OOM fix ? bytes never assemble on control-plane disk) while an incremental hashlib.md5 accumulates; upload_complete returns {path, video_id(=streamed md5 hex), size_bytes}. The SPA (khelsutra/web) then DISCARDS that video_id (only logs it) and submits POST /api/process with just {video_path, locale, compute} ? ProcessRequest has no video_id field. The submit path re-derives the md5 cheaply via one GCS metadata stat (#484), probes the DB/bucket cache, atomically claims the job, and on a miss the drain worker (_execute_processing) downloads the ENTIRE gs:// video back to CP scratch, re-hashes it with compute_video_id, and runs the full validator suite ON THE CP before dispatching DETECT to the L4 Cloud Run Job with --skip-validation (#513). That CP-side fetch+re-hash+validate is exactly what #524 (a)/(b) targets: the streamed upload md5 is already trustworthy, and validation could ride the L4 worker which re-fetches and re-hashes the same bytes anyway.","key_files":[{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/api/uploads.py","role":"Chunked-upload router: init/part/complete/abort; the #480/#471 stream-to-bucket design","key_lines":"1-25: module docstring = the #480/#471 design (parts stream through open_writer, instance memory bounds a CHUNK not the video); 52-58: UploadInitRequest{filename,total_size_bytes}/UploadCompleteRequest{total_parts}; 61-77: _upload_cfg reads live backend.main.global_config.security; 94-110: _staging_target ? remote staging iff storage.input_backend!='local' AND input_root non-empty (same config /api/process resolves against, so the staged URI is submittable verbatim); 149-214: upload_init ? ext/size checks, remote: stage_uri=f\"{input_root}/uploads/{upload_id}/{name}\" + storage.open_writer + hashlib.md5() in _uploads state (168-195); NotImplementedError?silent local fallback (177-179); local: .partial-<id> under security.upload_dir + require_free_bytes 507 guard (196-209); 217-283: upload_part ? strict sequential index (409 at 225-229), remote path buffers the WHOLE part in a bytearray (237-250) then one run_in_threadpool(writer.write)+hasher.update (251-254); 413-overflow aborts (270-274); 286-335: upload_complete ? remote: writer.close() finalizes, returns {path: stage_uri, video_id: hasher.hexdigest(), size_bytes, next} (299-319; 300-303 states the streamed md5 IS compute_video_id's digest); local: os.replace + full compute_video_id re-read (320-335); 113-137: _abort_upload closes writer + deletes staged object"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/main.py","role":"/api/process endpoint + ProcessRequest model + CLI single-shot path","key_lines":"258-272: ProcessRequest fields = video_path, player_pool, max_frames, segmenter, locale, compute('local'|'cloud'|None), force ? NO video_id field; 904-974: process_video ? edge-auth (917), validate_input_path with allowed_root=security.allowed_video_root + allowed_uri_root=storage.input_root (921-939, bucket-URI tenant jail), resolve_input_ref (935), local-existence fast-fail on resolved key only (946), segmenter check (953), jobs_api.submit_process_job (964), drain kick via

_get_executor().submit(_drain_due_jobs) (971-973); 1492-1593: CLI mode mirrors the same acquire_local_input+compute_video_id+validate flow (1506-1516)"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/api/jobs.py","role":"The whole submit decision: #484 submit-time md5 + cache probe + #448/#481 atomic dedup claim","key_lines":"80-153: submit_process_job ? order is (1) submit_time_video_id (98), (2) probe_process_cache unless force (99-101), (3) remote-input scratch 507 guard only on probe miss (102-103), (4) atomic claim (104), (5) claim winner realizes cache hit ? mark_job_done instantly, no GPU (111-128); 29-68: claim_process_job ? dedup keyed on (owner_id, video_path) NOT video_id (37-40); video_id persisted at CREATE so a restarted CP can re-associate bucket outputs (38-40, CP-rebuild §4.2); 71-77: _fetch_scratch_guard (tempdir 507)"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/orchestration.py","role":"Pipeline execution: the redundant re-hash, CP-side validation, cloud dispatch","key_lines":"220-239: submit_time_video_id ? gcs-only, ONE metadata stat, md5Hash?hex, None for local/composite, never raises; 287-317: probe_process_cache ? DB then {output_root}/outputs/{video_id}/report.json; 320-395: cached_process_result ? per-video_id lock re-hydrate (bucket=truth, DB=cache); 366-368: legacy videos/<md5>/ source-recovery prefix; 741-763: THE RE-HASH ? notify('hashing') ? storage.acquire_local_input downloads the full gs:// video to CP scratch (748-750) ? video_id=compute_video_id(local_video) (751), even when the job row already carries the submit-time md5; 802-873: VALIDATION ON THE CP ? #518 verdict cache keyed (video_id, config_fingerprint) (814-843), registry.run_all_with_context (846), set_validation_cache (851-856), failure short-circuits before dispatch (858-873); 906-913: _maybe_dispatch_remote called AFTER validation passes; 423-705: _maybe_dispatch_remote ? compute picker (473-512, 'local' weights fast-fail 480-498), local-upload staging to gs://{input_root}/{video_id}/{basename} + req.video_path repoint + add_video upsert (523-579), out_root required (580-586), --set forwarding of indexing.skip_ai_handoff + wasb knobs (598-619), ComputeJobSpec(skip_validation=True) #513 (620-632), video_id divergence warn-and-keep-local (647-655), ingest+provenance (657-705); 920-923: in-process segmenter under _LOCAL_COMPUTE_LANE"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/api/job_worker.py","role":"Async drain loop (DB is the clock) + #471 restart-safe remote-job resume","key_lines":"261-284: _drain_due_jobs claims via db.claim_due_job; 145-193: _run_claimed_job ? _execute_processing, then set_job_video_id from result (177-178), mark_job_done; 122-142: _job_to_request rebuilds ProcessRequest from the job row (video_path only ? no video_id is threaded into execution); 56-69: _cloud_dispatch_possible; 86-101: configure_drain / deployment.max_concurrent_remote_jobs; 334-490: resume_interrupted_remote_jobs + _resume_one_remote_job ? polls outputs/<md5>/report.json using the persisted submit-time video_id, never re-dispatches (proof the md5-at-submit contract already carries recovery weight)"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/compute/cloud_run.py","role":"CloudRunCompute dispatch shim: argv contract, done-marker poll, #515 fast-fail","key_lines":"57-60: container forced to compute_target=local_gpu + detector_impl=native (never re-dispatches); 244-318: run_infer ? done_uri=f\"{out_root}/_dispatch/{token}.done\" (252), argv=['infer','--video-uri',?,'--out-uri',?,'--done-uri',?] +--segmenter +--skip-validation +--set k=v (253-269), bucket-poll with on_wait heartbeat (281-290), post-timeout rescue + cancel (291-295), outputs_uri=f\"{out_root}/outputs/{video_id}/\" from the marker's video_id (296-307); 88-101: execution_terminal_state advisory ? terminated-without-marker ? RemoteExecutionFailed fast-fail (#515); marker checked FIRST, stays sole success signal (320-345)"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/worker/__main__.py","role":"The L4 worker CLI (python -m backend.worker infer) ? where #524(a) validation would land","key_lines":"244-299: cmd_infer ? anti-recursion dispatch guard (254-256), PULL storage.fetch to scratch/in.mp4 (265-267), IDENTITY video_id=compute_video_id(local_video) (271 ? the worker's own authoritative re-hash), VALIDATE unless --skip-validation (273-291; 273-281 documents the 2026-07-07 threshold-divergence incident: CP passed min_blur_threshold=8.0, worker default 20.0 rejected after a successful GPU dispatch); 301-326: _fail pushes status='failed' report + done marker so polls terminate fast; 126-147: _serialize_and_push_outputs ? <out_uri>/outputs/<video_id>/{intervals.csv,report.json}; 218-241: _write_done_marker JSON {video_id, returncode, segments, status}; 722-756: argparse ? --video-uri, --out-uri, --done-uri, --skip-validation, --set"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/storage/__init__.py","role":"Storage facade: resolve/acquire/fetch/put/open_writer helpers","key_lines":"53-66: resolve_input_ref (input_backend/input_root resolution; ValueError?clean 400); 77-98: acquire_local_input ? local=identity passthrough, remote=download into tempfile.mkdtemp('rally-input-') (93-95); 101-119: cleanup_acquired_local_input; 233-245: open_writer facade (NotImplementedError?caller falls back to local staging); 170-197: fetch/put"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/storage/gcs.py","role":"GCS backend: streaming writer +

the md5-metadata bridge that makes #484 work","key_lines":"95-103: open_writer = blob.open('wb', ignore_flush=True), ~40MiB internal chunks, object never held in memory; 75-84: stat returns md5 via _md5_hex; 24-34: _md5_hex ? GCS md5Hash is base64 of the raw digest ? hex (== compute_video_id), composite objects ? None, malformed ? None never raised"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/utils/hashing.py","role":"Canonical video_id definition","key_lines":"23-33: compute_video_id = MD5 hex of the WHOLE file, streamed in 64KB chunks; single source of truth for the DB join key"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/config/models.py","role":"Typed config: every knob gating this flow","key_lines":"380-394: SecurityConfig ? allowed_video_root=None, upload_dir='output/uploads', max_upload_mb=4096, max_part_mb=95, allowed_upload_extensions=['mp4','mov']; 387-388: ? hosted mode must set allowed_video_root to contain upload_dir or /api/process rejects uploads; 413-440: StorageConfig ? output_root='', input_backend Literal[local|s3|gcs|gdrive|gdrive-api]='local', input_root=''; 454-473: DeploymentConfig ? compute_target Literal['','local_gpu','cloud_run','cpu_mock']='', max_concurrent_remote_jobs"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/khelsutra/web/src/api/client.ts","role":"SPA API client (sibling repo; vendored build served from backend/ui/assets/index-kJhH4dVE.js)","key_lines":"250-303: uploadInit/uploadPart(raw PUT bytes)/uploadComplete/uploadAbort; 315-344: uploadFile orchestrator ? init?sequential parts (chunked by server's max_part_mb, NaN-guarded fallback 95)?complete; 337: video_id from upload_complete is only LOGGED; 89-103: processVideo ? POST /api/process body = {video_path, locale, compute} ? video_id is never sent"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/khelsutra/web/src/hooks/useUpload.ts","role":"The exact point the streamed video_id is dropped","key_lines":"27: hook signature onComplete(path, filename) ? no video_id parameter; 97-99: onCompleteRef.current(res.path, file.name) ? res.video_id discarded"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/khelsutra/web/src/components/IngestPanel.tsx","role":"Upload?process handoff in the SPA","key_lines":"161-162: useUpload((path, filename) => submit(path, locale, sendCompute, filename)) ? submit = useIngestJob ? processVideo(video_path only)"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/api/static/app.js","role":"Legacy dev SPA (path text-box, no chunked upload)","key_lines":"88-95: POST /api/process {video_path, player_pool} from a typed local path ? irrelevant to the upload flow but shares the endpoint"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/utils/paths.py","role":"Input jail","key_lines":"73: validate_input_path(raw, allowed_root, allowed_uri_root) ? null-byte guard always; local paths jailed under security.allowed_video_root; bucket URIs jailed under storage.input_root"}],"flow":["1. SPA pre-flight: GET /api/capabilities advertises {max_upload_mb, max_part_mb} (uploads.py:80-91); client rejects oversize files before any network (useUpload.ts:62-66).","2. POST /api/upload/init {filename, total_size_bytes} ? ext/size checks; hosted (storage.input_backend!='local' + input_root set): server opens storage.open_writer on gs://<input_root>/uploads/<upload_id>/<filename> and creates a hashlib.md5() in the in-process _uploads dict (uploads.py:168-195); appliance: creates .partial-<upload_id> under security.upload_dir after a 507 free-space guard (196-209). Returns {upload_id, max_part_mb, next_part}.","3. PUT /api/upload/{id}/part/{i} with raw bytes, strictly sequential (409 on gaps). Remote: the part is buffered in RAM (? max_part_mb), then one run_in_threadpool(writer.write, data) + hasher.update(data) ? the assembled video never touches CP disk/tmpfs (uploads.py:233-254, the #480/#471 fix). Local: append to the partial file. Over-limit ? 413 + server-side abort.","4. POST /api/upload/{id}/complete {total_parts} ? part-count check; remote: writer.close() finalizes the GCS object and the response is {path: gs://<input_root>/uploads/<upload_id>/<name>, video_id: <streamed md5 hex ? identical to compute_video_id by construction>, size_bytes, next: 'POST /api/process with this path'} (uploads.py:299-319); local: os.replace into upload_dir (name-dedup suffix) + a full compute_video_id re-read of the file (320-335).","5. SPA: uploadFile resolves; useUpload.ts:99 passes ONLY (res.path, file.name) to IngestPanel ? submit ? client.ts processVideo POSTs /api/process {video_path: <gs URI or local path>, locale, compute}. The video_id from step 4 is logged (client.ts:337) and thrown away ? ProcessRequest (main.py:258-272) has no field to carry it.","6. POST /api/process (main.py:904-974): edge-auth tenant ? validate_input_path (bucket URI must be under storage.input_root ? the staged upload is, by construction) ? resolve_input_ref ? local-existence fast-fail (local only) ? segmenter registry check ? jobs_api.submit_process_job.","7. submit_process_job (jobs.py:80-153): orchestration.submit_time_video_id re-derives the md5 with ONE GCS stat (metadata md5Hash ? hex; works because GCS computes md5 server-side for the non-composite object the BlobWriter finalized) ? probe_process_cache (DB rows, else gs://<output_root>/outputs/<md5>/report.json) ?

remote-input scratch 507 guard on miss ? ATOMIC claim_or_create_process_job keyed (owner_id, video_path), video_id persisted on the row at CREATE ? cache hit completes the job instantly (202 {status:'done', cached:true}); miss returns 202 {status:'queued'} and kicks the drain.","8. Drain (job_worker.py:261-284 ? 145-193): _job_to_request rebuilds ProcessRequest (video_path only) ? orchestration._execute_processing: notify('hashing') ? storage.acquire_local_input DOWNLOADS the full gs:// video to a rally-input-* scratch dir on the CP (orchestration.py:748-750) ? video_id = compute_video_id(local_video) (751) ? the redundant re-hash #524(b) targets; on the cloud path this download exists only to feed hash + validation.","9. Validation ON THE CP (orchestration.py:802-873): #518 verdict cache (video_id + config_fingerprint match replays the stored verdict, no decode) else registry.run_all_with_context(local_video, config); failures write status='failed' and return BEFORE any dispatch ? the CP is the authoritative gate.","10. _maybe_dispatch_remote (orchestration.py:906 ? 423-705): compute picker (req.compute override vs deployment.compute_target); a LOCAL-path input gets staged UP to gs://<input_root>/<video_id>/<basename> and req.video_path repointed (523-579 ? N.B. a browser upload in hosted mode is already a gs:// URI so this branch is skipped); builds ComputeJobSpec(video_uri, out_uri=storage.output_root, segmenter, config_overrides, skip_validation=True per #513).","11. CloudRunCompute.run_infer (cloud_run.py:244-318): argv = ['infer','--video-uri',<gs>,'--out-uri',<gs>,'--done-uri', gs://<output_root>/_dispatch/<token>.done, '--segmenter',?,'--skip-validation','--set',?] with container forced to compute_target=local_gpu; bucket-polls the done-marker (sole success signal) with the #515 fast-fail when the execution terminates marker-less; reads marker {video_id, returncode} ? outputs_uri = gs://<output_root>/outputs/<video_id>/.","12. L4 worker (worker/__main__.py cmd_infer:244+): fetches the video AGAIN to its own scratch (265-267), re-hashes compute_video_id (271 ? third hash of the same bytes across the pipeline), SKIPS validation (282-291), segments on GPU, pushes outputs/<video_id>/{intervals.csv, report.json} (394), writes the done marker (415-417); failures push status='failed' artifacts + marker so polls terminate fast.","13. CP ingest (orchestration.py:645-705): warns if worker video_id != local video_id and keeps the LOCAL id (647-655); _ingest_remote_segments into the CP DB; _finalize_reingest destroy-after-confirm; response stamped with cloud_run provenance (execution name, outputs_uri); job_worker marks done + sets job.video_id."],"contracts":["upload_init request/response: POST /api/upload/init {filename, total_size_bytes?} ? {upload_id, max_part_mb, next_part}; parts are raw-bytes PUT /api/upload/{upload_id}/part/{index}, strictly sequential from 0 (409 otherwise); DELETE /api/upload/{upload_id} aborts.","upload_complete response: {path, video_id, size_bytes, next} ? hosted path = gs://<input_root>/uploads/<upload_id>/<filename> (inside the #465 input jail by construction), appliance path = abs local file under security.upload_dir; video_id = MD5 hex of the file bytes (streamed incrementally hosted; compute_video_id full re-read locally).","ProcessRequest (backend/main.py:258-272): {video_path: str, player_pool?, max_frames?, segmenter?, locale?, compute?: 'local'|'cloud', force: bool=false} ? NO video_id field; the job row later carries video_id (persisted at CREATE from the GCS-stat md5, '' when unknown).","video_id = MD5 hex of the whole file (backend/utils/hashing.py:23-33); GCS object metadata md5Hash (base64 raw digest ? hex via gcs.py:_md5_hex) equals it for non-composite objects ? the bridge that makes submit_time_video_id free; composite objects have NO md5Hash ? None.","gs:// layout: <input_root>/uploads/<upload_id>/<filename> (browser-streamed staging); <input_root>/<video_id>/<basename> (CP-staged local upload before cloud dispatch, orchestration.py:548); <output_root>/outputs/<video_id>/{report.json, intervals.csv} (worker results; report.json status=='success' is the cache/recovery truth); <output_root>/_dispatch/<token>.done (per-dispatch completion marker); <output_root>/videos/<md5>/ (legacy staged-canonical fallback for source recovery, orchestration.py:366-368).","Worker CLI: python -m backend.worker infer --video-uri <uri> --out-uri <prefix> --done-uri <uri> [--segmenter <name>] [--skip-validation] [--set k=v]*; container overrides always append deployment.compute_target=local_gpu + indexing.detector_impl=native (cloud_run.py:57-60).","Done-marker JSON (worker/__main__.py:218-241): {video_id, returncode, segments, status} ? missing returncode defaults to failure (cloud_run.py:297-299); marker presence is the SOLE success signal.","DB job row via claim_or_create_process_job(job_id, video_path, segmenter, player_pool, max_frames, owner_id, locale, compute, force, video_id) ? dedup key is (owner_id, video_path), video_id at CREATE enables restart recovery on outputs/<md5>/report.json (#471/P3-P4).","In-process upload state: module dict backend/api/uploads.py:48 _uploads[upload_id] = {filename, writer|tmp_path, hasher?, stage_uri?, config?, parts, bytes} ? lost on restart by design (single-box alpha; client re-inits).","Env vars for cloud dispatch capability: CLOUD_RUN_JOB + GOOGLE_CLOUD_PROJECT (+ storage.output_root) enable the compute='cloud' override on a default-local server (job_worker.py:56-69, orchestration.py:200-206); CLOUD_RUN_REGION for provenance."],"gotchas":["#480/#471 OOM fix: the 4GiB RAM-/tmp control plane OOM'd on a 2.68GB browser upload; the fix streams parts through

storage.open_writer so instance memory bounds a PART, not the video (uploads.py:1-25). BUT each part is still fully buffered in a bytearray (~95MB) before the single threadpool write (uploads.py:237-254) ? max_part_mb is the real memory bound; concurrent uploads multiply it.","The streamed md5 already IS compute_video_id's digest ? uploads.py:300-303 says so explicitly ? but the contract dead-ends: the SPA drops video_id at useUpload.ts:99 and ProcessRequest has no field for it. Today it is re-derived by a GCS stat at submit (#484, orchestration.py:220-239) and then the bytes are re-hashed TWICE more (CP drain at orchestration.py:748-751; L4 worker at worker/__main__.py:271). #524(b) = trust it end to end (either accept video_id in ProcessRequest or rely on the GCS-stat md5 and skip the drain's acquire+re-hash).","The drain's acquire_local_input (orchestration.py:748-750) downloads the ENTIRE gs:// video to CP scratch even on a cloud dispatch ? the download feeds only hash (751) + validation (846); DETECT re-fetches independently on the L4 (worker/__main__.py:265-267). This is the CP disk/CPU/latency cost #524(a) wants to eliminate by moving validation to the L4.","#513 validation-placement incident (2026-07-07): the worker's config is NOT fully forwarded, so its validators resolved min_blur_threshold=20.0 vs the CP's 8.0 and rejected a clip AFTER a successful paid GPU dispatch ? hence skip_validation=True on every dispatch (orchestration.py:620-632, worker/__main__.py:273-291). Any #524(a) design that turns worker validation back ON must forward the CP's resolved validation.* config (the --set override channel already exists, orchestration.py:598-619) or the divergence returns.","#518 validation-verdict cache: keyed (video_id, registry.config_fingerprint) in the CP DB (orchestration.py:814-856) ? an identical-content reupload skips the validator decode entirely; force=true bypasses. Moving validation to the L4 must decide where this cache lives (worker has only a scratch DB, worker/__main__.py:293).","#515: a worker execution that terminates WITHOUT writing the done-marker (crash/OOM/argparse-abort) is failed fast via the advisory Executions-API probe (cloud_run.py:88-101, 320-345) instead of burning the ~65-min poll budget; the marker stays the only success signal.","#484 submit-time md5 never raises and silently degrades: metadata stat failure / composite object / local input ? video_id='' on the row and the drain re-hashes. GCS BlobWriter-finalized objects are non-composite so browser uploads always have md5Hash ? but an object composed server-side (future stitch topology!) would NOT, breaking the free-stat assumption.","Dedup key mismatch: submit dedup is (owner_id, video_path), and every re-upload of the same bytes gets a fresh uploads/<upload_id>/ URI ? so the atomic claim NEVER dedups re-uploads; only the video_id cache probe (DB/bucket) prevents a second paid GPU run. An upload?stitch topology that changes URIs per attempt keeps this property.","video_id divergence on the cloud path (worker re-hash != CP hash) is warn-and-ingest-under-LOCAL-id (orchestration.py:647-655) ? if the CP stops hashing (#524b), the worker's marker video_id becomes the only late-binding check against the trusted upload md5.","open_writer NotImplementedError (e.g. mounted-Drive backend) silently falls back to LOCAL staging at init (uploads.py:177-179) ? the hosted no-local-disk invariant holds only for backends that stream (gcs does, base.py:66 default raises).","Upload state is in-process (uploads.py:48-49): a CP restart aborts all partial uploads (client re-inits); fine under the single-instance CP (maxScale=1, ADR D8) but a topology moving upload onto the L4 job breaks this assumption.","Hosted config coupling: security.allowed_video_root must contain upload_dir or /api/process 400s the uploaded LOCAL paths (models.py:387-388); bucket URIs are only accepted under storage.input_root (main.py:924-931). Local completion (appliance) re-hashes the whole file synchronously on the request threadpool (uploads.py:329).","No phase_placement knob exists anywhere in the repo today (grep: zero hits) ? #524(a) adds a NEW config field; nearest precedent knobs are deployment.compute_target (models.py:471) and the ComputeJobSpec.skip_validation flag (compute/base.py).","player_pool and max_frames are silently DROPPED on the cloud path (orchestration.py:597, 904-905 ? documented follow-up); a validation-on-L4 design widens the forwarded-config surface and should account for them.","Restart recovery (#471) leans on the submit-time video_id persisted at CREATE: resume_interrupted_remote_jobs polls outputs/<md5>/report.json and never re-dispatches (job_worker.py:334-490); forced jobs and explicit compute='local' rows are hard-failed instead (406-421). Removing the CP re-hash must keep the row's video_id populated for this to work."],"config_knobs":["storage.input_backend (Literal local|s3|gcs|gdrive|gdrive-api, default 'local') ? backend/config/models.py:432; selects remote upload staging when != 'local' with input_root set","storage.input_root (default '') ? models.py:433; the gs:// root uploads stage under and the /api/process bucket-URI jail","storage.output_root (default '') ? models.py:423; where the worker writes outputs/<md5>/ and _dispatch markers; required for cloud dispatch","security.upload_dir (default 'output/uploads') ? models.py:389; local-staging directory (app-home-resolved)","security.max_upload_mb (default 4096) ? models.py:392; total-size 413 cap + SPA pre-flight guard","security.max_part_mb (default 95) ? models.py:393; per-part cap = the real CP memory bound on the streaming path","security.allowed_upload_extensions (default ['mp4','mov']) ? models.py:394","security.allowed_video_root (default None) ? models.py:383; local-path jail for /api/process; must contain upload_dir in hosted mode","deployment.compute_target (Literal

''|local_gpu|cloud_run|cpu_mock, default '') ? models.py:471; cloud_run turns on dispatch; per-request ProcessRequest.compute ('local'|'cloud') overrides","deployment.max_concurrent_remote_jobs ? models.py:473 / job_worker.py:72-90; drain parallelism when cloud-capable","deployment.remote_poll_max_wait_sec (fallback 3900.0 in job_worker.py:431-434) ? resume/dispatch poll budget","indexing.skip_ai_handoff, indexing.wasb.decode_backend / batch_size / prefetch_batches ? forwarded to the worker as --set overrides (orchestration.py:598-619); the existing precedent channel for forwarding resolved CP config to the L4","phase_placement ? DOES NOT EXIST YET (zero grep hits); the knob #524(a) would introduce"]},{"summary":"The orchestration processing path is the control-plane (CP) pipeline for one video: a queued job row is drained by backend/api/job_worker.py, reconstructed into a ProcessRequest, and run through backend/orchestration.py::_execute_processing = fetch-to-local ? full-file MD5 re-hash ? validator suite (with a video_id-keyed verdict cache, #518) ? segmenter dispatch (in-process under a 1-slot local-compute lane, or a Cloud Run L4 Job via _maybe_dispatch_remote with skip_validation=True) ? ingest intervals.csv back into SQLite via the same add_play_segment writes ? destroy-after-confirm finalize ? observability report + scratch cleanup. _execute_compile is the sibling worker handler for reel stitching; _resolve_compile runs both at submit (fast 4xx) and in the worker (re-resolve). For issue #524: (a) validation today runs ONLY on the CP (the worker's own validation is explicitly skipped via --skip-validation after the #513 threshold-divergence incident); (b) the re-hash at orchestration.py:751 recomputes the MD5 the upload endpoint already streamed (uploads.py:316) and that the job row may already carry (persisted at claim, #484); (c) upload?stitch currently streams browser parts into the bucket, then the CP re-downloads the whole video to scratch just to hash+validate before dispatching the GPU job that downloads it a third time. There is no phase_placement knob anywhere in the repo yet (grep: 0 hits).","key_files":[{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/orchestration.py","role":"The whole processing/compile orchestration (extracted from backend.main, P23). Seam discipline: main's globals (global_config, db_instance, ProcessRequest, compute_video_id, emit_indexer_report, VideoCompiler) are resolved through `import backend.main as _main` at call time (lines 8-19, 721, 1122) so tests monkeypatching backend.main still intercept.","key_lines":"53: _LOCAL_COMPUTE_LANE = threading.BoundedSemaphore(1) (#466); 56-66: _finalize_reingest destroy-after-confirm; 108-155: _ingest_remote_segments (intervals.csv ? db.add_play_segment, metadata {source:'cloud_run', ending_reason, shot_count}, 20k row cap, hardened row parsing); 163: _WEIGHTS_BACKED_SEGMENTERS = {wasb_hybrid, tracknet_hybrid, fusion_hybrid}; 166-187: _local_detector_weights_present (env WASB_WEIGHTS ? indexing.<det>.weights_path); 190-206: _cloud_available; 220-239: submit_time_video_id (gs:// metadata md5 only, one stat, None for local/composite); 287-317: probe_process_cache (read-only DB-or-bucket probe); 320-395: cached_process_result (DB=cache, bucket=truth re-hydrate under per-video_id lock, #485/#492); 423-705: _maybe_dispatch_remote ? 445-468 _failed (prunes NEW rows id>watermark, honest path=cloud_run|not_run provenance), 473-498 compute picker + the hosted-CP weights guardrail for compute='local', 506-512 get_compute_backend (None ? in-process default-OFF), 517-521 resolve_input_ref, 523-579 local-upload staging: put to <input_root>/<video_id>/<basename> (548,553) then repoint req.video_path AND db.add_video at the staged URI (575-579), 580-586 output_root required, 598-619 forwarded --set overrides (indexing.skip_ai_handoff; indexing.wasb.decode_backend/batch_size/prefetch_batches per #506/#507), 620-632 ComputeJobSpec with skip_validation=True (#513 rationale in comment), 638-644 compute.run_infer with on_wait heartbeat (#482), 645-646 rc!=0 ? failed, 647-655 worker-vs-local video_id mismatch ? WARN + ingest under LOCAL id, 657-670 _ingest_remote_segments, 672-676 watermark filter + _finalize_reingest, 679-705 provenance (job/region/execution/outputs_uri) + success response; 708-1002: _execute_processing ? 741-756 notify('hashing') ? storage.acquire_local_input (748) ? _main.compute_video_id(local_video) RE-HASH (751), cleanup-on-raise, 758-764 get_video/was_reingest, 767-770 seg_name = req.segmenter or indexing.default_segmenter, 781-798 whole-pipeline DB cache (processed+segments+not force), 802-856 VALIDATION: notify('validating'), registry.config_fingerprint (814), cached-verdict replay unless force (820-843), else registry.run_all_with_context(local_video, config) (846), db.set_validation_cache (851-856), 858-873 failures ? add_video 'failed' + failed response (no segmentation), 876-879 add_video 'processed', 881-893 defensive segmenter lookup, 900 segment_watermarks, 906-913 _maybe_dispatch_remote (returns ? cloud branch stamped response['compute']), 914-930 in-process: compute_actual='in_process', segmenter.process_video(video_id, local_video, req.player_pool, req.max_frames) under _LOCAL_COMPUTE_LANE (920-923), prune-on-raise (928), 932-942 Failure branch prunes NEW rows, 944-955 success finalize, 956-1002 finally: provenance stamp (961-977), emit_indexer_report with video_path=local_video + _report_config_for_input (982-997), storage.cleanup_acquired_local_input (1000-1002); 1026-1033 _CompileError(status_code, detail); 1036-1082 _resolve_compile (video exists, safe_output_filename, segment_ids match,

stitching.min_shot_count floor with unknown-count exemption); 1085-1114 reel_object_uri/_storage_settings/signed_reel_url; 1117-1204 _execute_compile ? params from job row (1125-1128), re-resolve (1131-1136), VideoCompiler with localized watermark (1144-1150), acquire_local_input(video['filepath']) (1159-1161), ffmpeg stitch under _LOCAL_COMPUTE_LANE (1164-1165), cleanup in finally (1175-1179), best-effort mirror to <output_root>/highlights/<video_id>/<name> (1185-1193), returns {status, filepath, download_url} (1197-1204)"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/api/job_worker.py","role":"Job pickup: the drain/wake loop that claims queued/due jobs and invokes _execute_processing/_execute_compile; also restart recovery for orphaned remote jobs (#471).","key_lines":"49-53: executor + _drain_max_workers (1 default); 56-83: _cloud_dispatch_possible/_resolve_drain_workers (deployment.max_concurrent_remote_jobs when cloud-capable; local work still serialized by _LOCAL_COMPUTE_LANE); 86-101: configure_drain (called once at startup, main.py:1635) + _get_executor; 122-142: _job_to_request ? rebuilds ProcessRequest from the job ROW (video_path, player_pool, max_frames, segmenter, compute, force from params JSON) ? NOTE: job['video_id'] is NOT threaded into the request today (the seam for #524b); 145-193: _run_claimed_job ? kind dispatch (compile vs process), db.set_job_video_id(result['video_id']) (177-178), mark_job_done (179), M8 cost + M11 flywheel hooks (181-186), JobWaiting park (187-190), crash ? mark_job_failed (191-193); 261-283: _drain_due_jobs (claim_due_job loop, DB-is-the-clock re-arm); 303-314: _arm_wake_timer; 334-490: resume_interrupted_remote_jobs/_resume_one_remote_job ? polls outputs/<md5>/report.json for md5-keyed RUNNING rows a restart orphaned; never re-dispatches; depends on video_id having been persisted AT SUBMIT (works only when submit_time_video_id resolved ? trusting the upload's streamed id would widen this coverage)"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/main.py","role":"FastAPI routes + startup wiring. ProcessRequest model; /api/process submit; /api/compile submit-side _resolve_compile.","key_lines":"258-272: ProcessRequest {video_path, player_pool, max_frames, segmenter, locale, compute, force} ? NO video_id field today; 904-974: POST /api/process ? edge auth (916-919), validate_input_path + resolve_input_ref ? 400 (929-939), local-existence fast-fail on input_ref.key (946-949), unknown-segmenter 400 (953-958), jobs_api.submit_process_job (963-966), drain kick only when queued (970-973); 1003-1022: GET /api/jobs/{id}; 1033-1035: uploads router include; 1041-1079: /api/highlights signed-URL 307 (#463); 1098-1141: POST /api/compile ? _resolve_compile at SUBMIT for fast 4xx (1113-1122), create_compile_job persists video['filepath'] as the job's video_path (1127-1135), drain kick; 1477: recover_stale_jobs, 1484: resume_interrupted_remote_jobs, 1635: configure_drain ? all in main() startup"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/api/jobs.py","role":"The whole /api/process submit decision: cheap md5 ? cache probe ? scratch guard ? atomic claim ? instant completion on cache hit (#448/#484/#492).","key_lines":"29-68: claim_process_job ? db.claim_or_create_process_job(..., video_id=video_id) ? the submit-time md5 IS persisted on the row when known; 80-153: submit_process_job ? orchestration.submit_time_video_id(input_ref, cfg) or '' (98), probe_process_cache gated on non-empty id + not force (99-101), _fetch_scratch_guard only on probe miss (102-103), claim (104-110), winner realizes cached_process_result + mark_job_done instantly (111-128), probe-hit-but-realize-failed fallback re-runs the scratch guard and fails the claimed row before raising (129-142)"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/api/uploads.py","role":"Chunked upload router ? the source of the ALREADY-STREAMED video_id that #524b wants /api/process to trust.","key_lines":"94-110: _staging_target (remote iff storage.input_backend != local AND input_root set); 168-195: upload_init remote path ? storage.open_writer to <input_root>/uploads/<upload_id>/<name>, state carries hashlib.md5() (185); 217-283: upload_part ? remote: buffered part ? writer.write + hasher.update(data) (250-254); local: append to .partial file (NO hasher ? local md5 is computed at complete); 286-335: upload_complete ? remote: returns {path: stage_uri, video_id: st['hasher'].hexdigest()} (314-319, 'the md5 accumulated per part IS compute_video_id's digest'); local: os.replace then video_id = compute_video_id(final_path) (326-334). Response says 'next: POST /api/process with this path' ? but the SPA sends only the path; the video_id is dropped on the floor at submit"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/utils/hashing.py","role":"compute_video_id ? canonical identity: MD5 of the WHOLE file, streamed in 64 KB chunks.","key_lines":"23-33: the single hash helper; on the CP re-hash of a bucket input this is preceded by a full-file download (acquire_local_input), which is the actual cost"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/storage/__init__.py","role":"Storage facade used by the pipeline: local-vs-remote input resolution, fetch-to-scratch, and cleanup.","key_lines":"53-74: resolve_input_ref/input_is_local (input_backend/input_root resolution); 77-98: acquire_local_input ? local ? identity (ref.key, no copy); remote ? tempfile.mkdtemp(prefix='rally-input-') + fetch (full download); 101-119:

cleanup_acquired_local_input ? deletes ONLY a 'rally-input-*' scratch dir (basename check), local identity paths untouched; 170-197: fetch/put"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/pipeline/validators/base.py","role":"Validator registry: run_all_with_context (the function _execute_processing calls) + config_fingerprint (the #518 cache key).","key_lines":"121-154: run_all_with_context ? shared frame-sampling pass then each validator; exceptions become failed ValidationResults; 156-181: _prepare_frame_samples (ONE decode pass for the 3 frame validators ? the minutes-long 4K cost on the CPU-only CP); 188-228: config_fingerprint ? sha256 of {schema:_VALIDATION_FINGERPRINT_SCHEMA(=1, line 23), sorted validator names, sport, full validation.* block}; 232: global `registry`. Registered set (backend/pipeline/validators/__init__.py:10-15): FormatValidator, ResolutionFPSValidator, PTSContinuityValidator, SceneCutDensityValidator, BlurValidator, RelevanceValidator"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/compute/base.py","role":"ComputeBackend ABC + the payload/result contracts.","key_lines":"34-51: ComputeJobSpec {video_uri, out_uri, segmenter, config_overrides tuple, skip_validation (default False ? the direct worker CLI still validates)}; 54-68: ComputeResult {returncode (0 ok / 2 validation-or-config / 3 segmenter / 4 timeout-or-bad-marker / 5 terminal-no-marker), outputs_uri, video_id, report, execution}"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/compute/__init__.py","role":"get_compute_backend factory ? the default-OFF seam.","key_lines":"42-97: only compute_target=='cloud_run' (or target_override='cloud_run') returns CloudRunCompute; policy.max_wait_sec derived from deployment.remote_poll_max_wait_sec when the caller didn't inject one (78-83); coordinates from env CLOUD_RUN_JOB / CLOUD_RUN_REGION (default asia-south1) / GOOGLE_CLOUD_PROJECT (85-87)"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/compute/cloud_run.py","role":"CloudRunCompute ? dispatch one L4 Job execution + bucket-poll the done-marker.","key_lines":"57-60: forced container overrides (deployment.compute_target=local_gpu, indexing.detector_impl=native ? never re-dispatch); 244-318: run_infer ? done_uri=<out>/_dispatch/<uuid>.done (252), argv = ['infer','--video-uri',?,'--out-uri',?,'--done-uri',?] + ['--segmenter',?] + ['--skip-validation' if spec.skip_validation] + ['--set',k=v]* (253-269), client.run_job (271), poll_until with on_tick=on_wait (282-290), marker carries video_id + returncode (missing rc defaults to 1) (296-306), outputs_uri=<out>/outputs/<video_id>/ (307), fetch report.json (308); 320-356: _poll_check ? marker first (sole success signal), Executions-API fast negative path raises RemoteExecutionFailed (#515); 375-420: _handle_poll_timeout ? one last marker rescue, else best-effort cancel + PolicyTimeout"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/worker/__main__.py","role":"The L4 worker CLI (`python -m backend.worker infer ?`) ? the code that would HOST validation under #524a.","key_lines":"244-256: cmd_infer ? load_config + _apply_overrides(args.set), startup checks (rc 2), get_compute_backend guard (dispatch only if its OWN config says cloud_run ? disarmed in the container); 260-262: scratch = tempfile.mkdtemp('rally-worker-') (never deleted ? the Job instance is ephemeral); 264-271: PULL input from bucket then compute_video_id RE-HASH on the worker (271) ? this stays the authoritative content check even if the CP trusts a request id; 273-291: VALIDATE ? the SAME validator_registry.run_all_with_context, SKIPPED when --skip-validation (282-291; the #513 incident is documented at 276-280: passed CP at min_blur_threshold=8.0, worker default 20.0 rejected post-dispatch); 292-299: per-scratch throwaway Database + add_video; 301-326: _fail ? pushes status='failed' report.json + intervals.csv then the done-marker with the real rc (validation failure = rc 2, marker written ? the CP poll terminates fast); 339-355: segmenter run (NOTE: player_pool not passed; max_frames only via --max-frames); 379-392: emit_indexer_report in finally; 394-418: push outputs/<video_id>/{intervals.csv,report.json} + SUCCESS done-marker {video_id, returncode, segment_count, status}; 754: --skip-validation argparse flag; 194: _dispatch_remote threads skip_validation into the spec"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/infrastructure/database_media.py","role":"DB writes the pipeline uses for videos/segments + the #518 validation cache.","key_lines":"17-40: add_video UPSERT (never cascades ? a failed re-validation can't destroy prior segments); 42-95: set/get_validation_cache ? table validation_cache(video_id PK, fingerprint, passed, results JSON, created_at), one latest verdict per video (DB migration v10); 97-113: segment_watermarks (MAX id per table); 115-152: prune_segments (*_upto = drop OLD on success, *_after = drop NEW on failure); 154-182: add_play_segment (the ONE writer both in-process segmenters and cloud ingest use)"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/infrastructure/database_jobs.py","role":"Jobs table ops.","key_lines":"228-313: claim_or_create_process_job ? atomic INSERT?WHERE NOT EXISTS keyed (owner_id, video_path) over active states; persists video_id at CREATE (240, 260, cols line 272-275) ? '' ? the drain fills it via set_job_video_id after the re-hash; force always inserts; 120-125: set_job_video_id; 127: mark_job_done; 508: claim_due_job; 628: interrupted_remote_process_jobs (md5-keyed running

process rows for restart recovery)"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/config/models.py","role":"Typed config ? where a phase_placement knob would live (DeploymentConfig).","key_lines":"24-43: ValidationConfig (min_resolution 1280x720, min_fps 29.9, min_blur_threshold 20.0, max_scene_cuts_per_minute 0.5, pts_max_gap_seconds 10.0, relevance.*); 413-451: StorageConfig (backend, output_root, input_backend, input_root, workspace knobs, per-backend dicts); 454-483: DeploymentConfig ? profile ''|truly-local|cloud-serving|cpu-mock (468), compute_target ''|local_gpu|cloud_run|cpu_mock (471), max_concurrent_remote_jobs default 4 ge1 le64 (477), remote_poll_max_wait_sec default 3900.0 (483). NO phase_placement field exists anywhere in the repo"}],"flow":["SUBMIT: POST /api/process (backend/main.py:904) ? edge-auth ? validate_input_path + storage.resolve_input_ref (400 on bad scheme) ? local-existence fast-fail ? jobs.submit_process_job (backend/api/jobs.py:80): submit_time_video_id (gs:// md5Hash metadata stat only, orchestration.py:220-239; None for local paths and composite objects) ? probe_process_cache (read-only) ? 507 scratch guard only on probe miss ? claim_or_create_process_job persisting video_id when known (database_jobs.py:240) ? cache-hit winner completes the job instantly via cached_process_result; otherwise 202 {job_id, status:'queued'} and the caller submits _drain_due_jobs to the executor (main.py:970-973)","PICKUP: job_worker._drain_due_jobs (job_worker.py:261) loops db.claim_due_job ? _run_claimed_job (145): kind='compile' ? _execute_compile, else _job_to_request (122; rebuilds ProcessRequest from the row ? video_id column NOT threaded in) ? _execute_processing(config, db, req, progress=set_job_progress)","ACQUIRE + RE-HASH (orchestration.py:741-756): notify('hashing') ? storage.acquire_local_input(req.video_path) ? identity for a local path, FULL download to a rally-input-* scratch dir for gs://(storage/__init__.py:77-98) ? video_id = compute_video_id(local_video) (751, full-file MD5) ? recomputed even when uploads.py already streamed the identical digest and the job row carries it","DB CACHE (orchestration.py:781-798): existing 'processed' video with segments and not force ? return the cached payload","VALIDATE ON THE CP (orchestration.py:802-856): fingerprint = registry.config_fingerprint(config) (814) ? unless force, replay a cached verdict from db.get_validation_cache when the fingerprint matches (#518, 820-843) ? else registry.run_all_with_context(local_video, config) runs the 6 registered validators (Format, ResolutionFPS, PTSContinuity, SceneCutDensity, Blur, Relevance ? validators/__init__.py:10-15) with one shared frame-decode pass ? db.set_validation_cache (851-856); any failure ? add_video 'failed' + a failed response, segmentation never runs (858-873)","PRE-SEGMENT (orchestration.py:876-900): add_video(video_id, req.video_path, 'processed', results) ? segmenter class lookup (seg_name = req.segmenter or indexing.default_segmenter) ? play/candidate watermarks captured for destroy-after-confirm","COMPUTE DECISION (orchestration.py:906 ? 423-705): _maybe_dispatch_remote ? req.compute 'local' forces in-process (guarded: a hosted CP without local detector weights fails fast, 480-498); 'cloud' forces cloud_run via _cloud_available + target_override; None ? get_compute_backend(cfg) (None for every non-cloud_run compute_target ? fall through in-process)","CLOUD DISPATCH: local upload ? staged up to <input_root>/<video_id>/<basename> via storage.put, req.video_path + the videos-table filepath repointed at the staged URI (523-579); payload = ComputeJobSpec{video_uri, out_uri=storage.output_root, segmenter, config_overrides=(indexing.skip_ai_handoff, indexing.wasb.decode_backend/batch_size/prefetch_batches when set), skip_validation=True} (598-632) ? CloudRunCompute.run_infer builds argv 'infer --video-uri ? --out-uri ? --done-uri <out>/_dispatch/<token>.done [--segmenter ?] --skip-validation --set ?' plus forced container overrides compute_target=local_gpu + detector_impl=native (cloud_run.py:57-60, 253-269) ? Cloud Run Job execution starts, CP bucket-polls the done-marker with the #482 progress heartbeat and the #515 Executions-API fast-fail","WORKER (worker/__main__.py:244-419): fetch input from bucket ? compute_video_id re-hash (271) ? validation SKIPPED under --skip-validation (282-291) ? segmenter.process_video (no player_pool on this path) ? push outputs/<md5>/intervals.csv + report.json (status success|failed, carries video_uri) ? done-marker {video_id, returncode, segment_count, status}; failures (rc 2/3) also push a failed report + marker via _fail so the CP poll terminates fast","INGEST BACK (orchestration.py:645-705): rc!=0 ? _failed (prunes NEW rows id>watermark, keeps prior good rows); worker video_id != local video_id ? WARN, ingest under the LOCAL id (647-655); _ingest_remote_segments fetches <outputs_uri>/intervals.csv and persists each row via db.add_play_segment with metadata {source:'cloud_run', ending_reason, shot_count} (108-155) ? new-rows filter by watermark ? _finalize_reingest drops the OLD rows ? success response with compute provenance {path:'cloud_run', job, region, execution, outputs_uri}","IN-PROCESS ALTERNATIVE (orchestration.py:914-946): compute_actual='in_process'; segmenter.process_video(video_id, local_video, req.player_pool, req.max_frames) runs under the 1-permit _LOCAL_COMPUTE_LANE (920-923; remote dispatches never hold it, so N remote polls overlap while local work stays serial); raise/Failure ? prune NEW rows; success ? _finalize_reingest","FINALLY (orchestration.py:956-1002): stamp response['compute'] provenance ({path: in_process|not_run, requested}) + the [COMPUTE] log line ? emit_indexer_report(video_path=local_video, config via _report_config_for_input so remote-input

reports land under output_base/reports not disposable scratch) ?
storage.cleanup_acquired_local_input deletes the rally-input-* scratch (local identity paths
untouched)","COMPLETION (job_worker.py:177-186): set_job_video_id(result['video_id']) ?
mark_job_done(result) ? M8 cost ledger + M11 flywheel hooks (both default-OFF); JobWaiting ?
park with next_poll_at; crash ? mark_job_failed","COMPILE: POST /api/compile runs
_resolve_compile at SUBMIT for fast 4xx (main.py:1113-1122) and persists the video's source
filepath on the job; the worker's _execute_compile RE-resolves (orchestration.py:1131 ? a row
can change between submit and run), acquire_local_input(video['filepath']) (1159 ? re-downloads
a bucket source to scratch), stitches under _LOCAL_COMPUTE_LANE (1164), best-effort mirrors the
reel to <output_root>/highlights/<video_id>/<name> for the signed-URL 307 (#463, 1185-1193),
cleans the scratch in finally"],"contracts":["ProcessRequest (backend/main.py:258-272):
{video_path: str, player_pool?: [str], max_frames?: int, segmenter?: str, locale?: str,
compute?: 'local'|'cloud'|None, force: bool=False} ? NO video_id field; adding one (or threading
job['video_id'] through _job_to_request, job_worker.py:122-142) is the #524b seam","jobs row
(database_jobs.py:272-275): id, video_path, video_id ('' until the drain re-hashes; pre-filled
at claim when submit_time_video_id resolved ? #484/CP-rebuild §4.2), segmenter, player_pool
JSON, max_frames, status queued|waiting|running|done|failed, kind NULL|'compile', policy,
owner_id, locale, compute, params JSON ({'force':true})","upload_complete response
(uploads.py:314-334): {path: <stage_uri or abs local path>, video_id: <md5 ? streamed hasher for
bucket staging, compute_video_id(final_path) for local>, size_bytes, next: 'POST /api/process
with this path'} ? the streamed md5 IS compute_video_id's digest (same bytes, same algorithm;
comment at 300-303)","Worker argv (cloud_run.py:253-269): python -m backend.worker infer
--video-uri <uri> --out-uri <output_root> --done-uri <output_root>/_dispatch/<uuid4>.done
[--segmenter <name>] [--skip-validation] --set k=v ? + forced --set
deployment.compute_target=local_gpu --set indexing.detector_impl=native","Done-marker JSON
(worker/__main__.py:229-234): {video_id, returncode, segment_count, status} at
<output_root>/_dispatch/<token>.done ? the SOLE success signal; missing returncode reads as 1
(cloud_run.py:299)","Bucket layout: <output_root>/outputs/<md5>/intervals.csv (columns
start,end,shot_count,ending_reason) + report.json (status, video_uri, ?);
<output_root>/highlights/<video_id>/<name> (mirrored reels); <output_root>/videos/<md5>/ (legacy
staged canonical, read-path fallback orchestration.py:366-368);
<input_root>/uploads/<upload_id>/<name> (streamed browser uploads);
<input_root>/<video_id>/<basename> (CP-staged local uploads,
orchestration.py:548)","ComputeJobSpec (compute/base.py:34-51) / ComputeResult (54-68): rc
semantics 0 ok, 2 validation-or-config, 3 segmenter failure, 4 poll timeout/bad marker, 5
terminated-without-marker","Env vars: CLOUD_RUN_JOB + GOOGLE_CLOUD_PROJECT (+ CLOUD_RUN_REGION,
default asia-south1) ? Cloud Run coordinates AND the _cloud_available/_cloud_dispatch_possible
capability signal; WASB_WEIGHTS (local weights override); DEPLOYMENT_PROFILE (preset
selection)","validation_cache table (database_media.py:42-95, DB v10): video_id PK, fingerprint
(registry.config_fingerprint = sha256 of schema-version + sorted validator names + sport + full
validation.* block), passed, results JSON ? one latest verdict per video","GET /api/jobs/{id}
(main.py:1003-1022): {job_id, status(execution state), progress, video_id, error,
result(pipeline verdict in result.status), reports, timestamps}"],"gotchas":["#524b impact ?
trusting the request/streamed video_id would change EXACTLY: (1) orchestration.py:741-756 ? the
notify('hashing') + acquire_local_input(748) + compute_video_id(751) block; for a gs:// input
the acquire is a FULL video download to CP scratch whose only pre-dispatch consumers are the
hash and the validators ? with a trusted id and worker-side validation, a pure-cloud run needs
NO CP fetch at all; (2) job_worker.py:122-142 _job_to_request currently DROPS job['video_id'] ?
the id already persisted at claim (database_jobs.py:240) never reaches _execute_processing; (3)
safety nets that remain: the worker independently re-hashes the bucket bytes
(worker/__main__.py:271) and orchestration.py:647-655 already handles worker-id != CP-id (warns,
ingests under the local id) ? under a trusted id that divergence check becomes the authoritative
content verification; (4) restart recovery (job_worker.py:334+) and the submit cache only work
when video_id is on the row ? a trusted upload id EXTENDS coverage to inputs where gs metadata
md5 is unavailable (submit_time_video_id docstring: composite objects carry no md5Hash; local
paths return None); (5) the emit_indexer_report finally-block uses local_video (989) ? if the
fetch is skipped, report emission needs a fallback path; (6) local in-process runs and
validators still need local_video, so the fetch can only be skipped when detection is remote AND
validation moved/skipped","#524a impact ? moving validation to the worker would change EXACTLY:
(1) orchestration.py:802-873 (the validating block incl. #518 cache read/write and the failure
short-circuit) becomes conditional on the new placement knob; (2) the skip_validation=True stamp
at orchestration.py:626-631 / ComputeJobSpec.skip_validation (compute/base.py:51) / '--skip-
validation' (cloud_run.py:264-267) flips meaning ? the worker gate at worker/__main__.py:282-291
becomes authoritative; (3) the #513 incident is the landmine: the worker Job applies NO
deployment profile and only receives the overrides orchestration.py:598-619 forwards

(skip_ai_handoff + 3 wasb knobs) ? its validation.* resolves to model DEFAULTS (min_blur_threshold 20.0 vs the CP's configured 8.0), so worker-side validation MUST forward the resolved validation.* config as --set overrides (or the whole fingerprint) or it re-introduces the observed passed-then-rejected failure; (4) a worker validation failure surfaces today only as rc=2 via the done-marker + a status='failed' report.json with NO ValidationResult list ? the CP's _failed branch (orchestration.py:645-646) would render 'cloud job failed (returncode 2)' instead of _validation_failure_message; the report.json schema would need to carry the results for parity with the CP response shape and db.add_video/validation_cache writes; (5) val_results feed 5 CP consumers: the response, db.add_video (861-865, 877-879), set_validation_cache (851-856), emit_indexer_report (992), and _maybe_dispatch_remote's staged-upload add_video (577-579); (6) the CP fingerprint (base.py:188-228) is computed from CP config ? caching a worker-produced verdict under it is only honest if the worker validated under the same effective config","#480 (the motivating waste): browser uploads STREAM into the bucket precisely to keep the 4 GiB RAM-/tmp CP from OOMing ? yet _execute_processing immediately re-downloads the whole video into CP scratch to hash+validate, and the L4 worker downloads it AGAIN for detection (3 transfers of the same bytes per cloud run)","#513: worker re-validation observed rejecting a clip the CP passed (min_blur_threshold 8.0 vs worker default 20.0) ? the reason skip_validation exists; any redesign that runs validation on the worker must solve config forwarding first","#515: a worker that dies before writing the done-marker is caught by the Executions-API fast negative path (cloud_run.py:320-356, RemoteExecutionFailed ? worker rc 5) instead of burning the ~65-min poll budget; the marker stays the sole SUCCESS signal","#518: the validation-cache replay leaves val_context={} ? the report's media block degrades to size-only (no fresh ffprobe_meta), a deliberate cost (orchestration.py:825-828)","Destroy-after-confirm everywhere: watermarks captured pre-run (orchestration.py:900); success prunes OLD rows (id<=wm), failure/empty prunes NEW rows (id>wm) ? _finalize_reingest (56-66), the segmenter-raise handler (924-929), the Failure branch (932-934), and _maybe_dispatch_remote._failed (448). Any redesign must keep failures from destroying prior good segments","add_video is an UPSERT, never INSERT OR REPLACE (database_media.py:24-31): REPLACE would cascade-delete play_segments ? a past incident; keep that invariant","_LOCAL_COMPUTE_LANE (orchestration.py:53) serializes in-process detection AND the compile ffmpeg stitch regardless of drain worker count (#466); the drain scales to deployment.max_concurrent_remote_jobs only when cloud dispatch is possible (job_worker.py:72-83). #524c ('upload?stitch on the L4') interacts with this: today the stitch is CP-local and lane-serialized, and _execute_compile re-downloads a bucket source to CP scratch (orchestration.py:1159)","_maybe_dispatch_remote MUTATES req.video_path when staging a local upload (orchestration.py:575) and repoints the videos-table filepath at the staged URI (577-579) so a later /api/compile can re-fetch after the instance recycles ? the jobs row keeps the ORIGINAL submitted path","player_pool and max_frames are silently DROPPED on the cloud path (orchestration.py:597, 903-905; worker call at __main__.py:348-350 passes only max_frames from its own CLI) ? a documented follow-up; a phase_placement design should not accidentally promise them","video_id-mismatch semantics (orchestration.py:647-655): the worker's outputs land at outputs/<WORKER-md5>/ but ingest happens under the LOCAL id ? result.outputs_uri is derived from the WORKER's id (cloud_run.py:307), so ingest actually reads the worker-id path while DB rows carry the local id; under a trusted request-id this is the divergence to alarm on, not merely warn","Worker scratch (tempfile.mkdtemp('rally-worker-'), worker/__main__.py:260) is never deleted ? acceptable only because the Cloud Run Job instance is ephemeral; CP scratch IS cleaned (rally-input-* dirs, cr-ingest-/cr-report-/cr-read- TemporaryDirectories)","Concurrency invariants to preserve: one re-hydrate per video_id (_REHYDRATE_LOCKS, orchestration.py:274-284, #492); the atomic INSERT?WHERE NOT EXISTS submit claim keyed (owner_id, video_path) not video_id (database_jobs.py:288-295) ? two different paths to the same content still create two jobs (the DB cache collapses the second at run time)","The CP is single-instance by spec (maxScale=1, D8) ? the in-process _uploads registry, the rehydrate locks, and process-wide semaphores all rely on this; an L4-primary topology (#524c) that adds a second compute plane must not assume those locks span machines (the bucket + SQLite serialization are the only cross-process coordination)","grep confirms NO phase_placement config exists yet anywhere; the natural home is DeploymentConfig (backend/config/models.py:454-483) beside compute_target, with presets.py parity (a unit test pins preset/profile parity per models.py:465)"],"config_knobs":["deployment.profile = '' ('' | truly-local | cloud-serving | cpu-mock) ? models.py:468; presets map profile?compute_target 1:1 (presets.py:45-75)","deployment.compute_target = '' ('' | local_gpu | cloud_run | cpu_mock) ? models.py:471; the ONLY value that arms remote dispatch is 'cloud_run' (compute/__init__.py:67)","deployment.max_concurrent_remote_jobs = 4 (ge 1, le 64) ? models.py:477; drain worker count when cloud-capable (job_worker.py:72-83)","deployment.remote_poll_max_wait_sec = 3900.0 ? models.py:483; must outlast the worker Job's --task-timeout (3600s)","storage.backend = 'local'; storage.output_root = '' (the bucket the worker writes to ? required for any cloud dispatch,

orchestration.py:580-586); storage.input_backend = 'local'; storage.input_root = '' (arms streamed uploads + local-upload staging) ? models.py:421-433","validation.min_resolution_width=1280 / min_resolution_height=720 / min_fps=29.9 / min_blur_threshold=20.0 / max_scene_cuts_per_minute=0.5 / pts_max_gap_seconds=10.0 / relevance.* ? models.py:24-43; ALL folded into the #518 fingerprint; NONE forwarded to the worker today","indexing.default_segmenter (fallback at orchestration.py:767-770); indexing.skip_ai_handoff + indexing.wasb.{decode_backend,batch_size,prefetch_batches} ? the only config forwarded to the worker (orchestration.py:598-619)","indexing.{wasb,tracknet}.weights_path + fusion.substrate ? the compute='local' weights guardrail (orchestration.py:166-187)","stitching.min_shot_count / begin_padding / end_padding / watermark ? _resolve_compile + _execute_compile (orchestration.py:1058-1081, 1138-1150)","security.upload_dir / max_upload_mb=4096 / max_part_mb=95 / allowed_upload_extensions ? uploads.py:61-91; security.allowed_video_root (hosted input jail, main.py:921-931)","billing.enabled=False (M8 cost ledger, job_worker.py:196-235); flywheel.enabled=False + hard-deny consent (job_worker.py:238-258)"]},{"summary":"The compute subsystem is the seam that decides WHERE the DETECT core runs. `backend/compute/` defines a `ComputeBackend` ABC + a lazy factory (`get_compute_backend`) that returns None (run in-process) for every `deployment.compute_target` except \"cloud_run\", in which case it returns `CloudRunCompute` ? a dispatch shim that starts a Cloud Run GPU Job execution of the SAME worker CLI (`python -m backend.worker infer ?`) with per-execution container-arg overrides, then bucket-polls a tiny JSON done-marker for completion (the marker, not the Cloud Run API, is the sole success signal; the Executions API is only an advisory fast-fail probe, #515). The worker entrypoint (`backend/worker/__main__.py`, the rally-worker Job's ENTRYPOINT) pulls the input from GCS, re-hashes it to md5 (`video_id`), optionally validates (skipped when the CP passes `--skip-validation`, #513), runs the segmenter, pushes `outputs/<video_id>/{intervals.csv,report.json}` and finally the done-marker. The control-plane side (`backend/orchestration.py::_maybe_dispatch_remote`) validates FIRST on the CP, dispatches with skip_validation=True, polls, then ingests intervals.csv into its SQLite DB. There is NO phase_placement knob today ? validation always runs on the CP (CPU) and only DETECT runs on the L4; the worker CAN validate (default-on for the bare CLI) but its validation verdict/reasons are never serialized back (report.json/marker carry no validator results or error message ? only status+returncode), which is the gap #524(a) must fill.","key_files":[{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/compute/base.py","role":"ComputeBackend ABC + the job/result dataclasses + the #515 exception","key_lines":"20-31: RemoteExecutionFailed (terminal-without-marker, #515); 34-51: ComputeJobSpec (video_uri, out_uri, segmenter, config_overrides tuple, skip_validation bool default False ? the #513 field, EXACT name `skip_validation`); 54-68: ComputeResult (returncode 0/2/3 mirrors worker, 4 = poll timeout/unreadable marker; outputs_uri, video_id, report dict, execution name for provenance); 71-87: run_infer(spec, on_wait) contract ? blocks until outputs durable"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/compute/__init__.py","role":"Factory: the default-OFF seam selecting local-inline vs cloud dispatch","key_lines":"42-97: get_compute_backend ? returns None unless target==\"cloud_run\" (67-68); target_override for the per-request SPA picker (62-66); 78-84: poll budget derived from deployment.remote_poll_max_wait_sec when caller passed DEFAULT_POLICY; 85-88: env coordinates CLOUD_RUN_JOB (default \"rally-worker\"), CLOUD_RUN_REGION (default \"asia-south1\"), GOOGLE_CLOUD_PROJECT"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/compute/cloud_run.py","role":"CloudRunCompute: dispatch shim + bucket-poll + #515 fast-fail","key_lines":"57-60: _DEFAULT_CONTAINER_OVERRIDES (--set deployment.compute_target=local_gpu + indexing.detector_impl=native ? anti-recursion); 63-101: CloudRunJobClient ABC (run_job/cancel_execution/execution_terminal_state advisory probe); 104-189: GoogleCloudRunJobClient (real SDK, lazy-import; container args override per execution at 135-142); 192-211: _classify_execution_state (pure; completion_time!=None => terminal); 244-318: run_infer ? done_uri = <out_root>/_dispatch/<uuid4>.done (250-252), argv build incl. --skip-validation (253-269), poll_until w/ on_tick heartbeat (282-290), marker->rc/video_id (296-307, missing returncode defaults to 1), fetch outputs/<vid>/report.json (308); 320-356: _poll_check ? marker FIRST, then Executions-API probe, race re-check, raise RemoteExecutionFailed (#515); 375-420: _handle_poll_timeout ? one post-timeout marker rescue else best-effort cancel + re-raise PolicyTimeout; 423-446: _read_marker/_read_json (empty marker dict still terminates poll, 430)"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/worker/__main__.py","role":"The worker entrypoint that runs INSIDE the rally-worker Cloud Run Job (also the bare CLI)","key_lines":"244-256: cmd_infer head ? load_config + --set overrides + startup checks; get_compute_backend guard (254-256) so a cloud target dispatches instead of running (174-214: _dispatch_remote maps RemoteExecutionFailed?rc 5, PolicyTimeout?rc 4); 264-268: PULL via storage.fetch ? '[worker] pulled' log (268); 271: re-hash compute_video_id(local_video); 282-291: #513 skip ? `if getattr(args, 'skip_validation', False)`

prints 'validation SKIPPED' and sets val_results,val_context=[],{} else
validator_registry.run_all_with_context; 301-326: _fail(rc) ? push status='failed'
report.json+intervals.csv THEN done-marker w/ real rc; 334-338: validation FAILED ? _fail(2);
340-346: unknown segmenter ? _fail(2); 352-354: segmenter Failure ? _fail(3); 91-114:
_write_report_json fields = {video_id,status,segment_count,video_uri,segments[{start,end}]} ? NO
error/validation fields; 117-150: _serialize_and_push_outputs ? <out_uri>/outputs/<video_id>/;
217-241: _write_done_marker = {video_id,returncode,segment_count,status}, best-effort never
raises; 394-419: success push + rc=0 marker; 739-760: argparse --done-uri + --skip-validation
flags"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/orchestration.py","role":"Control-plane: validate ? dispatch ? poll ? ingest;
also the #524(b) re-hash","key_lines":"423-705: _maybe_dispatch_remote ? picker local/cloud
(473-512); local-upload staging to <input_root>/<video_id>/<basename> + db repoint (523-579);
output_root required (580-586); config forwarding = ONLY indexing.skip_ai_handoff +
indexing.wasb.{decode_backend,batch_size,prefetch_batches} (598-619; validation.* NOT
forwarded); ComputeJobSpec(..., skip_validation=True) (626-632); run_infer + heartbeat
(638-641); exception ? _failed('cloud dispatch error', ran=True) (642-644); rc!=0 ?
_failed('cloud job failed (returncode N)') ? verdict detail LOST (645-646); worker-vs-CP
video_id mismatch warning, ingest under local id (647-655); _ingest_remote_segments (663-670);
provenance dict (679-694). 746-756: acquire_local_input + _main.compute_video_id re-hash (the
#524(b) redundant re-hash ? downloads a gs:// input to CP scratch just to hash+validate);
800-856: #518 validation-verdict cache (registry.config_fingerprint / db.get_validation_cache /
db.set_validation_cache, force bypass); 858-873: CP validation failure returns status='failed'
WITHOUT dispatch; 108-155: _ingest_remote_segments ? intervals.csv ?
db.add_play_segment(metadata source='cloud_run'), 20k row cap; 220-239: submit_time_video_id
(gs:// metadata md5, zero bytes); 287-317/320-379: probe_process_cache / cached_process_result
(bucket=truth; only report status=='success'
counts)"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/api/uploads.py","role":"Chunked upload that ALREADY streams the md5 (the
video_id #524(b) wants to trust)","key_lines":"168-195: init ? remote staging writer to
<input_root>/uploads/<upload_id>/<name> + hashlib.md5() hasher; 252-254: hasher.update per part;
299-319: complete returns {path: stage_uri, video_id: streamed md5, size_bytes} ? 'the md5
accumulated per part IS compute_video_id's digest'; 320-335: local-staging fallback hashes the
assembled file"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/api/jobs.py","role":"Submit hook: cheap-md5 cache probe before
queueing","key_lines":"80-146: submit_process_job ? submit_time_video_id (98) ?
probe_process_cache (99-101) ? atomic claim/dedup (104-110) ? realize cache hit instantly
(111-128) ? else queue; job row stores video_id when
known"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/api/job_worker.py","role":"Async job drain ? how failures surface to the job
today","key_lines":"145-193: _run_claimed_job ? job.status is EXECUTION state only: 'done' even
when the pipeline verdict is failed (verdict lives in result['status'], 149-152);
mark_job_failed ONLY on a raised exception (191-193); 122-142: _job_to_request rebuilds
ProcessRequest incl. compute picker; 261-278: _drain_due_jobs
loop"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/infrastructure/execution_policy.py","role":"Bounded poll primitives used by the
bucket-poll","key_lines":"45-75: ExecutionPolicy defaults (poll_every_n_sec=10,
max_wait_sec=1800 ? overridden to deployment.remote_poll_max_wait_sec=3900 by the factory);
41-42: PolicyTimeout; 152-201: poll_until (check() None=keep waiting; on_tick heartbeat
swallowed on error)"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/config/models.py","role":"DeploymentConfig ? the config surface #524(a)'s
phase_placement would join","key_lines":"454-483: profile (468), compute_target Literal
''|local_gpu|cloud_run|cpu_mock (471), max_concurrent_remote_jobs default 4 (477),
remote_poll_max_wait_sec default 3900.0 deliberately > the job's --task-timeout 3600 (478-483).
NO phase_placement knob exists today"},{"path":"C:/Users/avidu/Projects/khelsutra-
guru/badminton-highlight-indexer/deploy/cloudrun/Dockerfile","role":"rally-worker Job
image","key_lines":"115-119: env WASB_PYTHON/WASB_REPO_DIR/WASB_WEIGHTS +
RALLY_DETECTOR_IMPL=native + PYTHONUNBUFFERED=1 (live logs); 121-123: ENTRYPOINT
['python3','-m','backend.worker'] ? dispatcher supplies ALL per-execution args; no
DEPLOYMENT_PROFILE set, so the worker never applies the cloud-serving preset
itself"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/deploy/cloudrun/README.md","role":"Job creation contract","key_lines":"32-36: gcloud run
jobs create rally-worker ? --max-retries 0 --task-timeout 3600; 104: CP env exports
CLOUD_RUN_JOB/CLOUD_RUN_REGION/GOOGLE_CLOUD_PROJECT"}],"flow":["1. UPLOAD: SPA chunk-upload
(backend/api/uploads.py) streams parts straight into
gs://<storage.input_root>/uploads/<upload_id>/<name> while accumulating an md5 hasher per part;

POST /api/upload/{id}/complete returns {path: stage_uri, video_id: <streamed md5>} (uploads.py:299-319). This md5 IS compute_video_id's digest ? the value #524(b) wants /api/process to trust.","2. SUBMIT: POST /api/process ? backend/api/jobs.py::submit_process_job ? resolves a cheap md5 via gs:// object metadata (orchestration.submit_time_video_id, one stat, no bytes), probes the DB+bucket cache, atomically claims/dedups the job row, else queues it (the ProcessRequest body does NOT carry the upload's video_id ? it's re-derived).","3. DRAIN: job_worker._drain_due_jobs ? _run_claimed_job ? _execute_processing (orchestration.py:708+). notify('hashing'): storage.acquire_local_input DOWNLOADS the gs:// staged input to CP scratch (storage/__init__.py:77-98) and compute_video_id re-reads all bytes (orchestration.py:748-751) ? the redundant re-hash.","4. CP VALIDATION (the authoritative gate): orchestration.py:800-856 ? #518 verdict cache keyed (video_id, validator-config fingerprint) skips the decode on identical-content reupload; else registry.run_all_with_context on the LOCAL copy. Failure ? status='failed' response, never dispatched.","5. DISPATCH DECISION: _maybe_dispatch_remote (orchestration.py:423) ? per-request compute picker ('local' guarded by the weights-present check, 'cloud' guarded by _cloud_available), else server default via get_compute_backend(cfg) (None unless deployment.compute_target=='cloud_run'). A CP-local upload is staged up to <input_root>/<video_id>/<basename> first (523-579).","6. ENCODING THE DISPATCH: CloudRunCompute.run_infer (cloud_run.py:244-273) ? PURE ARGV, no env, no GCS manifest: overrides the rally-worker Job's container args for ONE execution to `infer --video-uri gs://? --out-uri gs://<output_root> --done-uri gs://<output_root>/_dispatch/<uuid4>.done [--segmenter <name>] --skip-validation --set indexing.skip_ai_handoff=? [--set indexing.wasb.*=?] --set deployment.compute_target=local_gpu --set indexing.detector_impl=native` via run_v2.RunJobRequest.Overrides (non-blocking start).","7. WORKER RUN (inside the L4 Job): backend/worker/__main__.py::cmd_infer ? get_compute_backend returns None (forced local_gpu) ? inline path; storage.fetch pulls the video to scratch ('[worker] pulled', line 268); re-hashes md5 (271); #513: args.skip_validation ? validation SKIPPED with empty val_results, else full validator suite (282-291); segmenter runs; SUCCESS ? push <out_uri>/outputs/<video_id>/{intervals.csv, report.json} then write the done-marker rc=0 (394-419); FAILURE ? _fail(rc) pushes a status='failed' report + marker with rc 2 (validation/unknown-segmenter) or 3 (segmenter Failure) (301-354).","8. CP POLL: poll_until every 10s up to deployment.remote_poll_max_wait_sec (3900s) on the done-marker object; each poll ALSO probes the Executions API ? terminal-state-without-marker ? RemoteExecutionFailed immediately (#515 fast-fail, cloud_run.py:320-356); PolicyTimeout ? one final marker rescue then best-effort cancel_execution + re-raise (375-420).","9. RESULT: marker {video_id, returncode, segment_count, status} ? fetch outputs/<video_id>/report.json ? ComputeResult(returncode, outputs_uri, video_id, report, execution). rc!=0 ? _failed('cloud job failed (returncode N)') ? no verdict detail; rc==0 ? _ingest_remote_segments reads intervals.csv into the CP DB ? _finalize_reingest (destroy-after-confirm) ? success payload with compute provenance {path:'cloud_run', job, region, execution, outputs_uri}.","10. JOB TERMINALITY: _run_claimed_job marks the job 'done' with the pipeline's result dict (verdict in result['status']); 'failed' job status only when _execute_processing RAISES (job_worker.py:145-193). RemoteExecutionFailed/PolicyTimeout are caught inside _maybe_dispatch_remote (642-644) so they surface as a done job with status='failed', message='cloud dispatch error: ?'."],"gotchas":["#513 origin incident (2026-07-07, comments at worker __main__.py:273-281 + orchestration.py:620-625): CP validated a 5312x2988 clip at validation.min_blur_threshold=8.0, worker re-validated at its default 20.0 and REJECTED after a successful GPU run. Root cause: the worker Job has NO DEPLOYMENT_PROFILE and gets only the explicitly forwarded --set overrides (orchestration.py:598-619) ? validation.* config is NOT forwarded today. If #524 moves validation ONTO the worker, this asymmetry reverses: worker-side validation would run at model-default thresholds unless the CP forwards validation.* overrides (or the phase move ships config forwarding).","#515 deployment coupling: an OLD worker image argparse-rejects an UNKNOWN flag and dies before writing any marker ? RemoteExecutionFailed (rc 5 path / 'cloud dispatch error'). Adding any new worker CLI flag for #524 (e.g. a validate phase flag) requires the image rollout BEFORE the CP starts sending it, or every dispatch fast-fails. The fast-fail itself is advisory-only: GoogleCloudRunJobClient.execution_terminal_state degrades to None on any error, so the marker stays the sole success signal (cloud_run.py:88-101, 358-373).","The done-marker write is BEST-EFFORT (never raises, __main__.py:240-241): a failed marker push on an otherwise-Succeeded execution surfaces as RemoteExecutionFailed ('Succeeded with NO completion marker'). A missing 'returncode' key in a present marker defaults to 1 = failure (cloud_run.py:297-299); a present-but-empty marker still terminates the poll (430).","NO validation verdict/error channel exists today: _write_report_json fields are exactly {video_id, status, segment_count, video_uri, segments} and the marker adds only returncode ? the worker's validation failure REASONS (printed at __main__.py:334-338) are never serialized. The CP collapses every non-zero rc to 'cloud job failed (returncode N)' (orchestration.py:645-646). ComputeResult.report is ALREADY fetched and returned (cloud_run.py:308-318) but orchestration ignores it ? the natural, additive carrier for

a worker validation verdict: add e.g. 'validation': [...ValidationResult.model_dump()...] / 'error': str to report.json (+ optionally the marker) and have _maybe_dispatch_remote read result.report on rc==2. Also note: a worker 'failed' report is treated as no-result by probe_process_cache (status!='success' ? re-dispatch on retry), and the #518 verdict cache is CP-local SQLite (db.set_validation_cache) ? a worker-side verdict would NOT populate it without new plumbing.","#518 (validation verdict cache): keyed (video_id md5, registry.config_fingerprint(config)); ANY validation config change misses; force=true bypasses; cache faults degrade to real validation (orchestration.py:800-856). Composes with #513 so a post-failure reupload reaches segmentation in ~0 validation time ? a #524 phase move must not regress this.","#480-class silent failure (wf294, worker _fail docstring __main__.py:301-308): worker failures MUST push a status='failed' report.json, not just a marker ? the CP's bucket-truth cache treats a missing/failed report as 'no usable result'. Any new worker phase must keep this invariant.","video_id identity cross-check: the worker re-hashes the pulled bytes; if worker md5 != CP md5 the CP warns and ingests under the LOCAL id (orchestration.py:647-655). If #524(b) drops the CP re-hash and trusts the upload's streamed md5, the worker's re-hash becomes the only byte-level verification ? and outputs land at outputs/<worker md5>/ while the CP looks under outputs/<CP md5>/ via marker.video_id, which is the WORKER's value (cloud_run.py:296-307), so the paths stay consistent even on divergence.","submit_time_video_id only works for gcs backend + non-composite objects (gs:// metadata md5Hash, orchestration.py:220-239); local files return None and keep deferred drain hashing ? a #524(b) design must handle the None case.","Poll budget vs task-timeout: remote_poll_max_wait_sec (3900) deliberately ABOVE the job's --task-timeout (3600) so the worker's own timeout stays authoritative (models.py:478-483); the old 1800s default abandoned healthy ~22-min 1080p60 runs mid-flight. rc=4 (timeout) triggers best-effort cancel_execution so the L4 stops billing.","Concurrency: max_concurrent_remote_jobs=4 lets remote bucket-polls overlap, but _LOCAL_COMPUTE_LANE (orchestration.py:47-53) serializes in-process segmentation + ffmpeg stitch to 1 ? relevant to #524(c)'s upload?stitch-on-L4 topology since the stitch currently competes for the single CP lane. Dispatch tokens are uuid4 per run_infer so concurrent jobs to one bucket never collide on _dispatch/<token>.done (cloud_run.py:235-237).","The worker's scratch SQLite DB (__main__.py:293-299) is throwaway ? the ONLY worker?CP channel is bucket artifacts (report.json / intervals.csv / done-marker). The dispatched container is forced inline via --set deployment.compute_target=local_gpu (never recursive re-dispatch)."],"contracts":["Dispatch encoding = container-args override on the existing Cloud Run Job (run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)), NO env vars, NO GCS manifest: `infer --video-uri <gs://?> --out-uri <gs://output_root> --done-uri <gs://output_root>/_dispatch/<uuid4-hex>.done [--segmenter <name>] [--skip-validation] --set <k=v> ?` (cloud_run.py:253-269; Dockerfile ENTRYPOINT `python3 -m backend.worker`).","#513 skip-revalidation flag: ComputeJobSpec.skip_validation (bool, default False, backend/compute/base.py:51) ? CLI flag `--skip-validation` (appended at cloud_run.py:264-267; parsed at worker __main__.py:753-760; consumed via getattr(args,'skip_validation',False) at 282). Set True by the CP at orchestration.py:631.","Done-marker (bucket object at <out_root>/_dispatch/<token>.done, written LAST by the worker): JSON {\"video_id\": <md5>, \"returncode\": int, \"segment_count\": int, \"status\": \"success\"|\"failed\"} (__main__.py:229-234). Missing returncode reads as 1; presence of the object terminates the poll.","Results manifest report.json (at <out_root>/outputs/<video_id>/report.json): {\"video_id\", \"status\": \"success\"|\"failed\", \"segment_count\", \"video_uri\" (the source the detection ran on, #485), \"segments\": [{\"start\": float, \"end\": float}]} (__main__.py:91-114). NO error-message or validation-verdict fields today. intervals.csv columns: start,end,shot_count,ending_reason (74-88); ingested via db.add_play_segment with metadata {source:'cloud_run', ending_reason, shot_count} (orchestration.py:108-155).","Worker exit codes: 0 ok; 2 validation-or-config (incl. unknown segmenter, startup-check fail); 3 segmenter failure; dispatcher-side only: 4 poll timeout (PolicyTimeout), 5 execution terminated without marker (RemoteExecutionFailed) (base.py:54-59; __main__.py:174-214).","Upload-complete response (uploads.py:314-319 / 330-335): {\"path\": <stage_uri or abs local path>, \"video_id\": <md5 ? streamed hasher for bucket staging, compute_video_id for local>, \"size_bytes\", \"next\": \"POST /api/process with this path\"}. ProcessRequest (main.py:258-272) carries video_path/segmenter/compute/force/locale ? NOT video_id (the #524(b) drop would add trust of the upload's value).","CP env vars (control-plane process): CLOUD_RUN_JOB (default 'rally-worker'), CLOUD_RUN_REGION (default 'asia-south1'), GOOGLE_CLOUD_PROJECT (required for real dispatch). Worker image env: WASB_PYTHON, WASB_REPO_DIR, WASB_WEIGHTS, RALLY_DETECTOR_IMPL=native, PYTHONUNBUFFERED=1 (Dockerfile:115-119).","Bucket layout under storage.output_root: outputs/<video_id>/{intervals.csv, report.json}; _dispatch/<token>.done; videos/<md5>/ (staged-canonical fallback). Staged uploads: <storage.input_root>/uploads/<upload_id>/<name> (streaming upload) or <storage.input_root>/<video_id>/<basename> (CP-local file staged at dispatch, orchestration.py:548).","ComputeResult.report = the parsed report.json dict, already surfaced to

the CP (cloud_run.py:308-318) but currently unread by orchestration ? the ready-made slot for a validation verdict round-trip."],"config_knobs":["deployment.profile (Literal ''|'truly-local'|'cloud-serving'|'cpu-mock', default '') ? backend/config/models.py:468; preset 'cloud-serving' sets compute_target='cloud_run' (backend/config/presets.py:52)","deployment.compute_target (Literal ''|'local_gpu'|'cloud_run'|'cpu_mock', default '') ? models.py:471; the ONLY value that activates the CloudRunCompute backend is 'cloud_run' (backend/compute/__init__.py:67)","deployment.remote_poll_max_wait_sec (float, default 3900.0) ? models.py:483; consumed at backend/compute/__init__.py:78-84 to stretch DEFAULT_POLICY.max_wait_sec (base poll cadence poll_every_n_sec=10.0, execution_policy.py:50)","deployment.max_concurrent_remote_jobs (int, default 4, ge=1 le=64) ? models.py:477; drain parallelism cap for concurrent cloud dispatches","storage.output_root ? required non-empty for any cloud dispatch (orchestration.py:580-586); the bucket the worker writes outputs+marker to","storage.input_root / storage.input_backend ? where uploads stream/stage; a cloud input_root is required to stage a CP-local upload for dispatch (orchestration.py:533-546)","indexing.skip_ai_handoff, indexing.wasb.decode_backend, indexing.wasb.batch_size, indexing.wasb.prefetch_batches ? the ONLY config forwarded to the worker as --set overrides (orchestration.py:598-619); validation.* (e.g. validation.min_blur_threshold) is NOT forwarded","ProcessRequest.compute ('local'|'cloud'|None per request) ? main.py:266-269, mapped to target_override='cloud_run' at orchestration.py:506","NO phase_placement knob exists anywhere today ? #524(a) would introduce it into DeploymentConfig alongside compute_target"]},{"summary":"backend/storage/ is the URI-driven, fully SYNCHRONOUS pluggable storage layer (COMPUTE_DECOUPLED_SERVING M2+). A thin facade (backend/storage/__init__.py) lazily constructs one of five StorageBackend implementations (local, s3, gcs, gdrive=mounted-Drive-over-local-FS, gdrive-api) from StorageConfig, keyed by URI scheme. It is the seam every pipeline stage crosses: browser uploads stream INTO the bucket via open_writer (with an incremental MD5 that already yields video_id for free ? #480/#484 groundwork for #524(b)); the control plane resolves inputs via resolve_input_ref and materializes them via acquire_local_input (identity for local, download-to-fresh-scratch for remote); the cloud dispatch stages local uploads UP with a blocking storage.put; the L4 worker pulls with storage.fetch and pushes outputs/done-marker with per-file storage.put; compile re-fetches the source and mirrors the reel with a best-effort blocking put. There is NO async/background copy primitive anywhere ? the only 'durable while bytes arrive' mechanism is GCS open_writer (streaming BlobWriter), which the upload path already exploits; an 'async copy-to-bucket during segmentation' model would need a new background-put primitive (or reuse of the existing upload-time streaming) plus completion/failure semantics.","key_files":[{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/storage/base.py","role":"StorageBackend ABC ? the whole backend contract","key_lines":"23-31: fetch_to_local (make object a local file; identity for local); 34-41: put (upload/copy local file to ref); 44-54: exists/stat/list/delete abstract; 56-64: signed_url default raises NotImplementedError (ttl=None defers to backend signed_url_ttl, #469); 66-75: open_writer default raises NotImplementedError ? writable file-like streaming bytes to ref as written, finalized on close() (#480, the anti-OOM streaming-upload primitive)"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/storage/__init__.py","role":"Facade: URI helpers + lazy backend construction (NOT the Registry-singleton idiom ? a name-switch in _backend_class so cloud SDKs import lazily)","key_lines":"53-66: resolve_input_ref(raw, storage_config) ? StorageRef under input_backend/input_root; ValueError for unsupported scheme (callers map to 4xx). 69-74: input_is_local. 77-98: acquire_local_input ? local ? identity return of ref.key (NO copy, byte-for-byte today's behavior); remote ? tempfile.mkdtemp(prefix='rally-input-', dir=scratch_parent) then fetch to <scratch>/<name or in.mp4>; cleans its own scratch on fetch exception. 101-119: cleanup_acquired_local_input ? no-op for local input; deletes the parent dir ONLY if its basename starts with 'rally-input-'. 122-146: _backend_class lazy resolve (local|s3|gcs|gdrive|gdrive-api). 149-167: get_storage ? settings from config[backend] with hyphen?underscore tolerance ('gdrive-api' reads pydantic field gdrive_api); **overrides merge; constructors accept **_ignored (unknown keys silently dropped). 170-245: module-level fetch/put/exists/signed_url/list_uris/open_writer ? each parse_uri + get_storage per call (stateless, no client reuse across calls)"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/storage/ref.py","role":"Pure-stdlib value objects + URI grammar","key_lines":"26-38: StorageStat(size, modified, etag, content_type, md5) ? md5 is whole-object HEX MD5, same value compute_video_id produces (#484); only GCS non-composite objects fill it; None = 'unknown' never 'no match'. 41-53: StorageRef(backend, bucket, key, uri); local/gdrive/gdrive-api use bucket='' with key=path; .name property. 56-77: parse_uri ? no scheme (incl. Windows C:\\ paths) ? local; gs:// normalized to gcs://; unsupported scheme ? ValueError. 80-111: resolve_input_uri precedence: explicit scheme > anchored local path (OS-

independent check, lines 100-104) > join under input_root > prefix with input_backend > raw unchanged"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/storage/local.py","role":"Default backend ? today's behavior byte-for-byte","key_lines":"20-28: base_dir resolution for relative keys; 30-41: fetch_to_local identity (dst honored only if different path ? shutil.copyfile; existing dst + overwrite=False returned as-is); 43-48: put = copyfile (no-op when src==dst); no signed_url, no open_writer (both raise)"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/storage/gcs.py","role":"GCS backend ? richest backend; the hosted-alpha input/output store","key_lines":"24-34: _md5_hex converts GCS base64 md5Hash ? hex so it equals compute_video_id (#484). 56-66: fetch_to_local REQUIRES dst; existing dst + overwrite=False short-circuits (trusts whatever is there); download_to_filename (whole-file, blocking). 68-70: put = upload_from_filename (whole-file, blocking, no multipart tuning). 75-84: stat fills md5. 95-103: open_writer ? blob.open('wb', ignore_flush=True), streams per ~40MiB internal chunk; THE only streaming-write primitive in the layer. 105-157: signed_url ? key-based, or keyless IAM signBlob fallback on Cloud Run/GCE metadata creds; #475: needs a FRESH cloud-platform-scoped credential (storage client's devstorage token is rejected) and roles/iam.serviceAccountTokenCreator on the runtime SA itself"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/storage/s3.py","role":"S3-compatible backend (AWS/R2/Spaces/MinIO via endpoint_url)","key_lines":"23-45: ctor (endpoint_url|AWS_ENDPOINT_URL_S3, region, addressing_style, signed_url_ttl; creds from boto3 chain only). 47-55: fetch_to_local requires dst, download_file. 57-59: put = upload_file. 69-76: stat ? NO md5 field (etag only). 90-96: presigned URL. NO open_writer ? hosted upload staging on s3 falls back to local disk (uploads.py:177-179)"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/storage/gdrive.py","role":"gdrive:// = alias over a locally MOUNTED Google Drive; delegates all I/O to LocalStorage","key_lines":"68-86: _autodetect_mount_root (macOS CloudStorage glob; Windows G:\\My Drive then letter scan). 89-101: ctor raises RuntimeError when unmounted ? headless boxes (Cloud Run) can NEVER use this backend. 110-116: _require_present raises remedy-rich FileNotFoundError for unsynced paths. 123-129: fetch_to_local identity-to-mount-path or copy-out. No signing, no open_writer"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/storage/gdrive_api.py","role":"gdrive-api:// = headless Drive-API backend (ADC service account) for the Cloud Run round-trip (M12)","key_lines":"105-141: _resolve_id path?file-id walk (create_dirs for put parents; duplicate (name,parent) returns first match). 150-179: fetch_to_local ? chunked MediaIoBaseDownload (32MiB, line 43) into dst+'.part' then os.replace (crash-safe, never whole-object in RAM). 181-207: put idempotent ? existing file overwritten in place via files().update (old bytes survive until new upload succeeds). 212-232: stat puts Drive md5Checksum into ETAG, NOT the md5 field. No open_writer, no signed_url"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/storage/capabilities.py","role":"Secret-free 'which media can THIS box use' block for GET /api/capabilities (Studio medium picker)","key_lines":"67-79: _bucket_configured (backend/input_backend match, non-empty per-backend dict, or a URI-shaped input_root/output_root/workspace_root). 82-90: bucket usable = SDK importable AND configured; free_gb None. 120-138: storage_capabilities ? {default_medium:'local', backends:[local,gcs,s3,gdrive]}"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/config/models.py","role":"StorageConfig ? every knob the layer reads","key_lines":"413-451: backend (default 'local'), output_root '', input_backend 'local', input_root '', single_folder_per_video False (per-video workspace, DEFAULT-OFF), workspace_root '', workspace_prefix 'workspaces', per-backend dicts local/s3/gcs/gdrive/gdrive_api"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/utils/workspace.py","role":"Pure path-math for the per-video workspace layout (Phase 0 ? helper exists, write sites NOT yet converged)","key_lines":"28-29: SUBDIRS=(source,detector,outputs,reels,logs,tmp), tmp is LOCAL_ONLY. 32-44: join_uri. 47-56: resolve_workspace_root precedence workspace_root?output_root?output.output_dir. 59-109: VideoWorkspace base=<root>/[prefix/]<video_id>. 112-121: local_scratch_base (always native FS)"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/api/uploads.py","role":"Chunked upload ? the producer of the streamed video_id that #524(b) wants /api/process to trust","key_lines":"94-110: _staging_target ? remote staging ONLY when input_backend != 'local' AND input_root non-empty. 169-195: init opens storage.open_writer on <input_root>/uploads/<upload_id>/<name>; NotImplementedError ? local staging fallback. 233-254: each part buffered (?max_part_mb) then writer.write via run_in_threadpool; hashlib.md5 accumulates incrementally. 299-319: complete ? writer.close() finalizes; returns {path: stage_uri, video_id: streamed-md5-hexdigest, size_bytes} ? 'the md5 accumulated per part IS compute_video_id's digest'. 320-334: local path ? os.replace then compute_video_id RE-READS the file. 113-136: abort ? close-then-delete (streaming writers cannot

cancel)"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/orchestration.py","role":"The storage layer's heaviest consumer: process pipeline, cloud dispatch stage-up, compile fetch + reel mirror","key_lines":"746-763: acquire_local_input + compute_video_id RE-HASH of the whole file (the #524(b) redundancy; for a bucket-staged upload = full re-download + full re-read on the control plane); cleanup on any exception. 1000-1002: cleanup in the outer finally. 516-579: _maybe_dispatch_remote ? local input + cloud input_root ? SYNCHRONOUS storage.put stage-up to <input_root>/<video_id>/<basename> (progress stage 'staging upload to bucket'), then repoint req.video_path + db.add_video at the staged URI; stage-ability classified by parse_uri(input_root).backend, NOT input_backend (lines 527-546). 108-150: _ingest_remote_segments ? fetch <outputs_uri>/intervals.csv into TemporaryDirectory('cr-ingest-'). 1153-1179: compile ? acquire_local_input(video['filepath']) re-fetches the FULL source from the bucket every compile; cleanup in finally. 1181-1193: best-effort blocking storage.put reel mirror to reel_object_uri (#463; failure = warn only). 1085-1092: reel_object_uri = <output_root>/highlights/<video_id>/<name>, '' unless output_root has '://'. 1095-1114: _storage_settings + signed_reel_url (exists + signed_url with active settings). 1005-1023: _remote_input_size_bytes ? backend.stat(ref).size for free-space pre-reserve, never raises"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/main.py","role":"API-boundary storage consumers","key_lines":"929-948: /api/process ? validate_input_path with allowed_uri_root=storage.input_root (#465 input jail: bucket URIs only under input_root) + resolve_input_ref; local-existence fast-fail on input_ref.key; NOTE ProcessRequest (258-272) has NO video_id field ? the upload's streamed md5 is discarded at this boundary. 545-588: persistent source cache <output>/source_cache/<video_id><ext> ? per-video locks, fetch to .tmp + os.replace (used only by /api/source). 712-717: _storage_settings dict. 780-901: /api/assets ? resolve/acquire, compute_video_id, storage.put into the chosen medium at <medium>/<video_id><ext>, cleanup in finally. 1497-1592: CLI single-shot ? acquire_local_input, compute_video_id, cleanup in finally"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/worker/__main__.py","role":"L4 Cloud Run Job worker ? bucket pull/push discipline (the box #524(a) wants validation moved to)","key_lines":"264-271: PULL storage.fetch(video_uri ? <scratch>/in.mp4) then compute_video_id re-hash ('pulled bytes must be byte-identical'). 273-279+: validation skipped when the dispatcher already validated (#513 ? control plane is the authoritative gate; worker's own config once wrongly re-rejected a passed video). 120-150: _push_outputs ? per-file blocking storage.put to <out_uri>/outputs/<video_id>/{intervals.csv,report.json}, shared by success AND failure paths. 217-241: _write_done_marker ? best-effort put of _done.json for the dispatcher's bucket poll. 260-262: scratch = args.scratch or tempfile.mkdtemp('rally-worker-') (ephemeral container FS)"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/flywheel.py","role":"Data-flywheel capture ? the only VideoWorkspace+storage.put consumer today","key_lines":"55-89: capture_job writes capture.json + optionally the source video into workspace.outputs/.source via storage.put (blocking); 44-52: consent gate is a hard-deny (always False) so nothing captures yet"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/compute/__init__.py","role":"Where region enters the picture (compute, not storage ? the storage layer itself is region-blind)","key_lines":"85-97: CLOUD_RUN_JOB (default 'rally-worker'), CLOUD_RUN_REGION (default 'asia-south1'), GOOGLE_CLOUD_PROJECT env ? CloudRunCompute"}],"flow":["UPLOAD (hosted, gcs input): POST /api/upload/init ? if storage.input_backend!='local' AND input_root set, storage.open_writer streams to gcs://<input_root>/uploads/<upload_id>/<name> (uploads.py:169-195); GCS BlobWriter pushes ~40MiB chunks as parts arrive (gcs.py:95-103). Otherwise a local .partial file under security.upload_dir.","UPLOAD parts: PUT /api/upload/{id}/part/{i} ? sequential; remote path buffers one part then writer.write in a threadpool hop, incremental hashlib.md5 update (uploads.py:233-254).","UPLOAD complete: writer.close() finalizes the object; response {path: staged gs:// URI (inside the #465 jail by construction), video_id: streamed md5 == compute_video_id digest, size_bytes} (uploads.py:299-319). Local path: os.replace into upload_dir then compute_video_id re-reads the file (320-334).","SUBMIT: POST /api/process ? validate_input_path(allowed_uri_root=input_root) + resolve_input_ref; local fast-fail stats input_ref.key; job row queued. ProcessRequest carries NO video_id, so the streamed identity is dropped here (main.py:258-272, 929-948).","CONTROL-PLANE EXECUTE (_execute_processing, orchestration.py:741-763): notify('hashing') ? acquire_local_input (LOCAL: identity, no copy; REMOTE: mkdtemp('rally-input-') + full download) ? compute_video_id RE-HASHES the whole local file ? the #524(b) redundant re-hash; for a gcs-staged upload this is a full re-download+re-read on the CPU control plane.","VALIDATE on the control plane against the LOCAL working copy (notify('validating'), orchestration.py:802+; verdict cached by video_id+config fingerprint per #518) ? this is the phase #524(a) wants moved behind a phase_placement knob onto the L4.","CLOUD DISPATCH (_maybe_dispatch_remote, orchestration.py:516-579): remote input dispatched as-is; a

LOCAL input is staged UP with a blocking storage.put to <input_root>/<video_id>/<basename> ('staging upload to bucket'), then req.video_path + the DB filepath are repointed at the staged URI (the local copy dies with the instance). Requires a URI-shaped input_root; output requires storage.output_root (580-586).","L4 WORKER (worker/__main__.py cmd_infer): storage.fetch(video_uri ? scratch/in.mp4) ? compute_video_id AGAIN (byte-identity check, 264-271) ? validation SKIPPED if --skip-validation (dispatcher validated, #513) ? segment ? _push_outputs per-file storage.put to <out_uri>/outputs/<video_id>/ (success AND failure both leave report.json) ? _write_done_marker put (best-effort).","RESULT INGEST (control plane): _ingest_remote_segments fetches intervals.csv from the bucket into a TemporaryDirectory and replays rows through db.add_play_segment (orchestration.py:108-150).","COMPILE (_run_compile, orchestration.py:1153-1193): acquire_local_input(video['filepath']) ? full re-fetch of the source from the bucket to fresh scratch ? ffmpeg stitch under _LOCAL_COMPUTE_LANE ? cleanup scratch in finally ? best-effort blocking storage.put of the reel to <output_root>/highlights/<video_id>/<name> (#463); /api/highlights then 307s to signed_reel_url (1103-1114).","SERVE (/api/source, main.py:614-671): resolve_input_ref; remote source fetched ONCE into the persistent <output>/source_cache/<video_id><ext> (tmp + os.replace under a per-video lock, 563-588); background thread warms an annotation proxy (thread is for ffmpeg, NOT a storage copy).","SCRATCH LIFECYCLE: every remote acquire_local_input creates a FRESH 'rally-input-*' dir (no cross-call caching ? each call re-downloads); the CALLER must invoke cleanup_acquired_local_input(raw, local_path, config), which deletes only dirs whose basename starts with 'rally-input-' and only for non-local raw inputs; acquire cleans its own scratch if the fetch itself raises. Worker scratch = 'rally-worker-*' mkdtemp on the ephemeral container FS (never explicitly deleted; container teardown reclaims it)."],"gotchas":["NO ASYNC COPY PRIMITIVE EXISTS. Every put/fetch is synchronous and whole-file (gcs upload_from_filename, s3 upload_file, local copyfile). The only concurrency anywhere near storage: uploads.py's per-part run_in_threadpool hop and main.py:666's proxy-warming thread (ffmpeg, not storage). For #524(c) 'async copy-to-bucket during segmentation' you must BUILD: a background-thread/executor put with completion tracking + failure semantics (is the job durable-blocked or best-effort?), or reuse the fact that a hosted browser upload is ALREADY streamed into the bucket at upload time via open_writer ? in that topology the durability copy predates segmentation and needs no new primitive; the gap is only the self-host/local-input case.","open_writer exists ONLY on GCSStorage (gcs.py:95-103). S3Storage/local/gdrive/gdrive-api raise NotImplementedError ? uploads.py:177-179 catches it and silently degrades to local-disk staging. Any design leaning on streaming-to-bucket is gcs-only today.","#484 primitive built but UNCONSUMED: StorageStat.md5 (hex, == compute_video_id) is filled by GCS stat (gcs.py:75-84) yet no caller outside backend/storage/ reads .md5 (grep-verified). It's the ready-made 'trust identity from metadata' hook for dropping the re-hash ? but only for GCS non-composite objects (None otherwise), and gdrive-api puts its md5Checksum in etag, not md5 (gdrive_api.py:230). uploads.py's streamed hexdigest (line 316) is the other, backend-independent identity source.","The re-hash being dropped in #524(b) is defense-in-depth today: orchestration.py:751 hashes the DOWNLOADED bytes, so a corrupted staged object or a URI swapped after upload_complete would be detected. Trusting the upload's video_id moves integrity to the bucket (GCS crc32c on upload) ? the worker still re-hashes after its own pull (worker/__main__.py:271).","fetch_to_local with an existing dst and overwrite=False returns dst WITHOUT checking completeness (gcs.py:62-63, s3.py:51-52) ? safe today only because acquire_local_input always downloads into a fresh mkdtemp and the source cache uses tmp+os.replace; gdrive_api additionally does .part+rename (gdrive_api.py:168-179). Don't add a reused fixed dst without adding atomicity.","#480 incident: the 4GiB RAM-/tmp control plane OOM'd assembling a 2.68GB browser upload ? that's why remote staging streams via open_writer and why anything that re-materializes a whole video on the control plane (e.g. the #524(b) re-hash download) is suspect.","#465 input jail: /api/process accepts bucket URIs only under storage.input_root (main.py:924-931, validate_input_path allowed_uri_root). Upload staging (<input_root>/uploads/<upload_id>/) and dispatch stage-up (<input_root>/<video_id>/) are inside the jail by construction. Any new upload?stitch-on-L4 topology must keep its staged objects under input_root or widen the jail deliberately.","#461 stage-up classification subtlety: whether a local input can be staged for cloud dispatch is decided by parse_uri(input_root).backend, NOT storage.input_backend (orchestration.py:527-546) ? an explicit gs:// input_root stages fine with input_backend left 'local'; an empty input_root cannot stage at all.","#513/#515/#518 lineage (directly relevant to moving validation to the L4): the worker SKIPS re-validation on cloud dispatch because the control plane validated with ITS config (a worker default once re-rejected a control-plane PASS, worker/__main__.py:273-279); verdicts are cached by video_id+config fingerprint (orchestration.py:802-818); a worker crash before the done-marker now fails fast (#515). Moving validation L4-side inverts #513's 'control plane is the authoritative gate' ? the config-fingerprint cache keying must move with it.","REGION/CO-LOCATION IS BROKEN FOR THE L4: the bucket (khelsutra-rally-corpus) is asia-south1/Mumbai (deploy/gcp/provision.sh:15,33; ADR-010 'co-located GPU + bucket'), but L4 Cloud

Run is region-limited ? deploy/cloudrun/README.md:20: 'use asia-southeast1 (Singapore), NOT asia-south1'; CLOUD_RUN_REGION defaults to asia-south1 (backend/compute/__init__.py:86). So today's worker fetch/push can be CROSS-REGION (egress + latency) ? a first-order input to any upload?stitch-on-L4 topology. The storage backends themselves are region-blind (GCS client has no region; s3 region is just a client setting, s3.py:28).","Failure semantics are heterogeneous by call site: dispatch stage-up put failure = clean job failure (orchestration.py:560-571); reel mirror put failure = warn-only, local file still served (1192-1193); done-marker put = best-effort never raises (worker 240-241); worker output push = raises (a push failure fails the job). An async-copy design must pick one of these postures explicitly.","Upload state is in-process, single-box (uploads.py:16-19): a server restart aborts partial uploads; a remote-staged abort deletes the finalized-partial object best-effort (close-then-delete ? streaming writers cannot cancel, 118-132). There is NO GC for orphaned gcs uploads/<upload_id>/ objects if the process dies between close() and /complete.","cleanup_acquired_local_input never raises: unsupported-scheme ValueError is swallowed (returns, __init__.py:110-114), and it refuses to delete anything not matching the 'rally-input-' basename ? so a caller passing a wrong path cannot delete user data, but forgetting to call it leaks a full video copy in temp.","Windows-correctness is deliberate: parse_uri treats 'C:\\...' as local (no '//', ref.py:60-65); resolve_input_uri's anchored-path check is OS- and Python-version-independent (ref.py:100-104 ? os.path.isabs on Windows py3.13 rejects '/data/x').","get_storage builds a FRESH backend (and SDK client) per call ? no pooling. Callers must thread the active settings dict (_storage_settings) or a non-default backend is silently constructed with defaults (bug class called out at orchestration.py:551-553, 1189-1191).","gdrive:// hard-requires a mounted Drive at construction (RuntimeError, gdrive.py:96-97) ? never usable on Cloud Run; the headless Drive path is gdrive-api:// (ADC service account; per-user OAuth is owner-gated, gdrive_api.py:14-17).","Per-video workspace (workspace.py + storage.single_folder_per_video, default False) is Phase-0: the layout helper exists and flywheel.capture_job uses it, but pipeline write sites have NOT converged ? a new topology should target this layout rather than inventing another key scheme."],"config_knobs":["storage.backend (default 'local') ? output-side backend selector","storage.input_backend (default 'local') + storage.input_root (default '') ? input resolution (resolve_input_uri precedence) AND the upload remote-staging trigger AND the #465 URI jail root","storage.output_root (default '') ? reel mirror + cloud dispatch out_root; must contain '://' to activate reel_object_uri","storage.workspace_root (default ''), storage.workspace_prefix (default 'workspaces'), storage.single_folder_per_video (default False, DEFAULT-OFF)","storage.local.base_dir (default ''), storage.s3.{endpoint_url,region,addressing_style='auto',signed_url_ttl=3600}, storage.gcs.{project,signed_url_ttl=3600,client(inject)}, storage.gdrive.mount_root (or GDRIVE_MOUNT_ROOT env), storage.gdrive_api.root_folder_id (default 'root')","security.upload_dir (default 'output/uploads'), security.max_upload_mb (4096), security.max_part_mb (95), security.allowed_upload_extensions (['mp4','mov']), security.allowed_video_root","deployment.compute_target ('', 'local_gpu', 'cloud_run', 'cpu_mock'; default ''), deployment.profile, deployment.max_concurrent_remote_jobs (4), deployment.remote_poll_max_wait_sec (3900)","env: CLOUD_RUN_JOB ('rally-worker'), CLOUD_RUN_REGION ('asia-south1' ? mismatched with the L4's actual asia-southeast1), GOOGLE_CLOUD_PROJECT, GOOGLE_APPLICATION_CREDENTIALS (ADC), AWS_ENDPOINT_URL_S3"],"contracts":["URI grammar (ref.py:56-77): bare path ? local; local://<path>; s3://<bucket>/<key>; gcs://<bucket>/<key> (gs:// normalized to gcs://); gdrive://<path-under-My-Drive-mount>; gdrive-api://<path-under-root_folder_id>. Unsupported scheme ? ValueError (API/CLI boundaries map to 4xx, never 500).","StorageRef(backend, bucket, key, uri) ? local/gdrive/gdrive-api: bucket='' and key=the path; .name = last path component.","StorageStat(size, modified, etag, content_type, md5) ? md5 is hex whole-object MD5 identical to compute_video_id output; filled only by GCS non-composite stat; gdrive-api reports Drive md5Checksum in etag instead.","upload_complete response: {path: <staged URI or abs local path>, video_id: <md5 hex ? streamed for remote staging, re-read for local>, size_bytes, next: 'POST /api/process with this path'} (uploads.py:314-335). ProcessRequest = {video_path, player_pool?, max_frames?, segmenter?, locale?, compute?('local'|'cloud'), force} ? NO video_id field today (main.py:258-272).","Bucket key layouts: staged uploads <input_root>/uploads/<upload_id>/<name>; dispatch stage-up <input_root>/<video_id>/<basename>; worker outputs <out_uri>/outputs/<video_id>/{intervals.csv,report.json}; done-marker at the dispatcher-supplied done_uri; reels <output_root>/highlights/<video_id>/<name>; per-video workspace <root>/[workspaces/]<video_id>/{source,detector,outputs,reels,logs,tmp} (tmp local-only).","acquire_local_input/cleanup pairing: caller owns the returned path's scratch for remote inputs; cleanup is keyed on the 'rally-input-' dir-basename prefix and is a no-op for local inputs; acquire self-cleans on fetch failure.","open_writer: write() streams bytes out incrementally, close() finalizes the object (durability point), flush() must be tolerated as a no-op; NotImplementedError is an EXPECTED signal meaning 'fall back to local

staging'.","signed_url: NotImplementedError expected (local always, gdrive); ttl=None defers to the backend's configured signed_url_ttl (#469); method 'GET'/'PUT'.","Credentials NEVER live in StorageConfig ? boto3 default chain / GCS ADC / Drive ADC supply them; settings dicts carry only endpoints/regions/paths/selectors.","get_storage settings lookup: config[<backend-name>] with hyphen?underscore fallback so 'gdrive-api' reads the pydantic field gdrive_api (__init__.py:160-165); unknown settings keys are swallowed by **_ignored ctors."]},{"summary":"CONFIG SYSTEM (backend/config/): a typed Pydantic AppConfig with load precedence \"built-in model defaults < deployment-profile preset < config.json < env vars < worker --set\" (presets.py:7-9). The ONLY placement knob today is deployment.compute_target (Literal \"\"|local_gpu|cloud_run|cpu_mock) ? consumed solely by get_compute_backend(), which returns a CloudRunCompute shim only for \"cloud_run\" and None (in-process) otherwise; it moves ONLY the detect phase. Validation and compile always run on the control plane. Config reaches the L4 rally-worker Job three ways: (1) image-baked env (RALLY_DETECTOR_IMPL=native, WASB_*), (2) per-execution CLI args built by CloudRunCompute (--set k=v overrides, --segmenter, --skip-validation, plus two FORCED disarm overrides deployment.compute_target=local_gpu + indexing.detector_impl=native), and (3) Job env vars that beat config (WASB_DECODE_BACKEND ? the no-redeploy rollback lever). The worker Job has NO DEPLOYMENT_PROFILE so presets never apply there; the control plane resolves config and forwards knobs as --set. The live control plane is configured by deploy/cloudrun/gen_service_yaml.py which base64-bakes a config.json into the service manifest wrapper (RALLY_APP_HOME=/tmp/apphome) + env DEPLOYMENT_PROFILE=cloud-serving/CLOUD_RUN_REGION/GOOGLE_CLOUD_PROJECT. A #521 phase_placement knob slots in as a new field on DeploymentConfig with canonical values pinned in presets.py and a parity test, exactly like compute_target.","key_files":[{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/config/models.py","role":"All typed config models; DeploymentConfig is the deployment section","key_lines":"454-483: DeploymentConfig ? profile (468, Literal ''|truly-local|cloud-serving|cpu-mock), compute_target (471, Literal ''|local_gpu|cloud_run|cpu_mock, default ''), max_concurrent_remote_jobs (477, default 4), remote_poll_max_wait_sec (483, default 3900.0). 202-295: IndexingConfig ? detector_impl (210, 'auto', env RALLY_DETECTOR_IMPL overrides), skip_ai_handoff (289), default_segmenter (203, 'yolo_hybrid'). 68-165: WasbIndexConfig ? decode_backend (131, 'cpu'), batch_size/prefetch_batches (118/121, None), device (91); nested-model validators (133-165) are the precedent for validating new knobs. 24-44: ValidationConfig (min_blur_threshold=20.0 etc. ? the #513 divergence source). 413-451: StorageConfig (backend/input_backend/input_root/output_root). 529-561: AppConfig root ? deployment field at 549; model_config extra='forbid' + validate_assignment=True at 554-557 (AppConfig ONLY ? nested models don't validate assignment)."},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/config/presets.py","role":"Profile presets + canonical enum values + env auto-detect (suggest-only, D15)","key_lines":"27-28: PROFILES + COMPUTE_TARGETS tuples (canonical values, mirrored by the models.py Literal, drift pinned by tests/test_config_deployment.py:77-97). 32-36: COMPUTE_TARGET_FOR_PROFILE 1:1 map. 43-80: PROFILE_PRESETS ? 'cloud-serving' (51-73) sets compute_target=cloud_run, detector_impl=native, skip_ai_handoff=False, wasb{batch_size:64, prefetch_batches:4, decode_backend:nvdec_resident}, storage.backend=gcs. 126-149: suggest_profile (K_SERVICE?cloud-serving, CI?cpu-mock; never auto-applies)."},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/config/loader.py","role":"load_config: defaults < preset < config.json merge; profile resolution","key_lines":"53-79: _resolve_explicit_profile ? DEPLOYMENT_PROFILE env WINS over config.json deployment.profile; unknown env value warns+falls through. 128-236: load_config ? config file is JSON at backend/utils/app_home.py::default_config_path() (RALLY_APP_HOME-rooted, else ./config.json); 193-219: preset deep-merge + re-stamp of profile and canonical compute_target (an active profile with blanked compute_target gets its canonical target restored ? 200-206). 39-50: _deep_merge. 82-105: _unknown_key_paths warning (unknown NESTED keys are silently dropped by Pydantic ? see WasbIndexConfig docstring models.py:69-74 for the past bug). 261-265: LIGHT_DEFAULT_CONFIG."},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/config/startup.py","role":"Per-profile boot coherence checks (M5) ? where a placement/target mismatch check belongs","key_lines":"240-297: cloud-serving branch ? STORAGE_INCOHERENT fatal (242-251), AI_HANDOFF_KEY_MISSING fatal (261-280), CREDS_UNCONFIRMED/CLOUD_CPU_DEVICE warns. 195-197: no profile ? strict no-op. 314-344: enforce_startup_checks (server calls with include_exposure=True at main.py:1241 and sys.exit(1) on fatal; worker via worker/__main__.py:153-171 returns rc 2)."},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/compute/__init__.py","role":"get_compute_backend ? THE consumption point of deployment.compute_target","key_lines":"62-68: target resolution (target_override param for per-request SPA 'cloud' choice; only 'cloud_run' returns a backend, everything else None=in-process). 78-83: remote_poll_max_wait_sec plumbed from config into ExecutionPolicy ? the cleanest precedent of a deployment.* field consumed at the seam. 85-88: Cloud Run coordinates

from env: CLOUD_RUN_JOB (default 'rally-worker'), CLOUD_RUN_REGION (default 'asia-south1'), GOOGLE_CLOUD_PROJECT."},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/compute/cloud_run.py","role":"CloudRunCompute ? how per-job config physically reaches the worker container","key_lines":"57-60: _DEFAULT_CONTAINER_OVERRIDES = ('deployment.compute_target=local_gpu', 'indexing.detector_impl=native') ? forced --set args that disarm recursive re-dispatch; a phase_placement knob needs the same disarm treatment. 253-269: argv build ? infer --video-uri/--out-uri/--done-uri [--segmenter] [--skip-validation] then --set for (*spec.config_overrides, *self._container_overrides). 135-141: real client passes argv as Cloud Run per-execution ContainerOverride args."},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/compute/base.py","role":"ComputeJobSpec/ComputeResult contracts","key_lines":"34-51: ComputeJobSpec ? config_overrides: tuple[str,...] of raw '--set k=v' strings (43-44); skip_validation: bool = False (45-51, the #513 field ? closest precedent for a phase-move flag). 54-68: ComputeResult rc semantics (0 ok / 2 validation-or-config / 3 segmenter / 4 poll-timeout; CLI adds 5 for RemoteExecutionFailed)."},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/worker/__main__.py","role":"Worker CLI ? how the container consumes config + overrides","key_lines":"245-246: cmd_infer loads config (defaults-only in the container; no config.json in the image) then _apply_overrides(config, args.set). 48-71: _coerce + _apply_overrides ? dotted-path setattr on the typed config (NOTE: nested models lack validate_assignment, so --set values on nested fields bypass Literal validation). 250-256: worker itself re-dispatches when compute_target=cloud_run (disarmed by the forced override). 282-291: --skip-validation branch. 746-760: infer argparse flags incl. --set and --skip-validation."},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/orchestration.py","role":"Control-plane phase sequencing + the --set forwarding pattern","key_lines":"741-856: _execute_processing ? hashing (748-751), then VALIDATION on the control plane (802-856, with the #518 verdict cache keyed by registry.config_fingerprint(config) at 814), then dispatch decision. 423-512: _maybe_dispatch_remote ? per-request compute='local'|'cloud' override handling; 506-512 get_compute_backend. 598-631: THE forwarding precedent ? control plane forwards resolved indexing.skip_ai_handoff + indexing.wasb.{decode_backend,batch_size,prefetch_batches} as --set overrides and sets skip_validation=True on the spec (the worker Job has no DEPLOYMENT_PROFILE so presets don't apply there ? comment at 603-608). 190-206: _cloud_available."},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/api/job_worker.py","role":"Drain concurrency + restart recovery read compute_target","key_lines":"56-69: _cloud_dispatch_possible (compute_target==cloud_run OR CLOUD_RUN_JOB+GOOGLE_CLOUD_PROJECT env + storage.output_root). 72-90: _resolve_drain_workers/configure_drain ? deployment.max_concurrent_remote_jobs consumed (#466); called at server startup main.py:1635. 359-361: resume path re-reads compute_target."},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/deploy/cloudrun/gen_service_yaml.py","role":"Live control-plane deploy: bakes config.json into the service manifest","key_lines":"62-87: build_config ? the baked config.json: deployment{profile:cloud-serving, compute_target:cloud_run}, indexing{skip_ai_handoff:true, default_segmenter:wasb_hybrid (#479)}, storage{gcs, output_root=gs://khelsutra-rally-serving, input_root=<bucket>/videos}, security pins (max_part_mb=24 #480). 92-102: wrapper base64-decodes it to /tmp/apphome/config.json + exports RALLY_APP_HOME. 129-137: env block ? DEPLOYMENT_PROFILE=cloud-serving, CLOUD_RUN_REGION, GOOGLE_CLOUD_PROJECT, RALLY_ALLOWED_HOSTS='*'. 28-29: CLOUD_RUN_JOB deliberately NOT set (reserved runtime env name ? manifest rejected; code default 'rally-worker' is right). 175-181: maxScale default 1 (#471 per-instance SQLite)."},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/deploy/cloudrun/Dockerfile","role":"Worker Job image ? env-baked config","key_lines":"115-119: ENV WASB_PYTHON/WASB_REPO_DIR/WASB_WEIGHTS + RALLY_DETECTOR_IMPL=native baked in. 121-123: ENTRYPOINT python3 -m backend.worker (dispatcher supplies all per-execution args). No config.json in the image; no DEPLOYMENT_PROFILE."},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/deploy/cloudrun/README.md","role":"Deploy runbook ? knob-flow doc + version-skew rules","key_lines":"60-65: 'How the knob reaches the worker' ? preset is single source of truth, forwarded as --set. 67-78: WASB_DECODE_BACKEND Job env = no-redeploy override/rollback lever (env beats forwarded --set). 79-90: version-skew warning ? a NEW worker CLI flag argparse-aborts an OLD image before the done-marker (roll worker-first; #513/#515). 32-39: rally-worker Job create flags (L4, --task-timeout 3600, GCS FUSE weights mount overriding the Dockerfile WASB_WEIGHTS). 99-148: control-plane wiring + §3a service-deploy flags."},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/pipeline/detectors/native_wasb_runner.py","role":"Env-over-config precedence at the leaf (worker side)","key_lines":"108-133: NativeWasbConfig.from_indexing_cfg ?

env(WASB_REPO_DIR/WASB_PYTHON/WASB_WEIGHTS/WASB_DEVICE/WASB_SCRATCH/WASB_DECODE_BACKEND) each beat the corresponding indexing.wasb.* config value."},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/tests/test_config_deployment.py","role":"Parity pin: model Literal == presets tuples","key_lines":"77-97: asserts set(PROFILE_PRESETS)==set(PROFILES) and the compute_target Literal == {'', *COMPUTE_TARGETS} ? a new placement enum must add the same parity assertions."}],"flow":["Control-plane boot (Cloud Run service rally-control-plane): the gen_service_yaml.py wrapper writes the baked config.json to /tmp/apphome and exports RALLY_APP_HOME + DEPLOYMENT_PROFILE=cloud-serving, then execs `python3 -m backend.main --server` (gen_service_yaml.py:97-102).","load_config() (backend/config/loader.py:128-236): built-in AppConfig defaults < cloud-serving preset (PROFILE_PRESETS, selected by the DEPLOYMENT_PROFILE env which wins over config.json deployment.profile) < the baked config.json, deep-merged; profile + canonical compute_target re-stamped (loader.py:196-219).","Server startup: enforce_startup_checks(config, include_exposure=True) ? fatal on cloud-serving incoherence (main.py:1241, sys.exit(1)); configure_drain(global_config) fixes drain workers at deployment.max_concurrent_remote_jobs when _cloud_dispatch_possible (main.py:1635; job_worker.py:56-90).","/api/process job drain -> orchestration._execute_processing: acquire_local_input + MD5 hash (748-751), VALIDATE on the control plane with ITS resolved config (802-856; #518 cache keyed by video_id + registry.config_fingerprint(config)), only a PASS proceeds.","_maybe_dispatch_remote (423-): per-request req.compute 'local'/'cloud' can override; else get_compute_backend(cfg) reads deployment.compute_target ? 'cloud_run' returns CloudRunCompute built from env CLOUD_RUN_JOB/CLOUD_RUN_REGION/GOOGLE_CLOUD_PROJECT with deployment.remote_poll_max_wait_sec as the poll budget (compute/__init__.py:62-97); None -> in-process segmenter (compile/stitch ALWAYS stays on the control plane).","Dispatch: orchestration builds config_overrides (--set indexing.skip_ai_handoff, indexing.wasb.decode_backend/batch_size/prefetch_batches ? forwarding the control plane's RESOLVED preset values because the worker has no profile) + skip_validation=True on ComputeJobSpec (598-632).","CloudRunCompute.run_infer: argv = ['infer', '--video-uri', ..., '--out-uri', ..., '--done-uri', <out>/_dispatch/<token>.done, '--segmenter', ..., '--skip-validation'] + --set for spec.config_overrides + the FORCED _DEFAULT_CONTAINER_OVERRIDES ('deployment.compute_target=local_gpu', 'indexing.detector_impl=native') so the container never re-dispatches (cloud_run.py:250-273); runs the Cloud Run Job execution with these as per-execution container-override args; bucket-polls the done marker with an Executions-API fast-negative probe (#515).","Worker container (ENTRYPOINT python3 -m backend.worker, env RALLY_DETECTOR_IMPL=native + WASB_* baked): cmd_infer -> load_config() (model defaults only ? no config.json, no profile) -> _apply_overrides(args.set) via dotted setattr -> startup checks are a no-op (no profile) -> get_compute_backend returns None (compute_target was forced to local_gpu) -> runs inline: pull, re-hash MD5, skip validation (--skip-validation), segment, push outputs/<video_id>/{intervals.csv,report.json}, write done marker (worker/__main__.py:244-419).","At the detection leaf, Job env vars beat the forwarded config: NativeWasbConfig.from_indexing_cfg reads WASB_DECODE_BACKEND/WASB_DEVICE/... first (native_wasb_runner.py:108-133) ? the operator's no-redeploy rollback lever.","Control plane ingests the bucket result under the pre-computed video_id and stamps cloud_run provenance (orchestration.py:645-705)."],"contracts":["Config file: a single JSON config.json at default_config_path() = $RALLY_APP_HOME/config.json when set, else ./config.json (backend/utils/app_home.py:30-56). No YAML, no .env-based config values (only env VAR overrides at consumption sites).","Env vars (control plane): DEPLOYMENT_PROFILE (selects preset, beats config.json), CLOUD_RUN_JOB (default 'rally-worker' ? must NOT be set on a Cloud Run service manifest, reserved name), CLOUD_RUN_REGION (default 'asia-south1' in code; live is asia-southeast1), GOOGLE_CLOUD_PROJECT (required for real dispatch), RALLY_ALLOWED_HOSTS, RALLY_APP_HOME.","Env vars (worker Job): RALLY_DETECTOR_IMPL=native, WASB_PYTHON/WASB_REPO_DIR/WASB_WEIGHTS baked in Dockerfile:115-119 (WASB_WEIGHTS re-pointed to the GCS FUSE mount at job-create, README:37); WASB_DECODE_BACKEND/WASB_DEVICE/WASB_SCRATCH optional overrides that beat any --set/config value (native_wasb_runner.py:117-132).","Worker CLI contract (the per-execution container args): `infer --video-uri <uri> --out-uri <prefix> [--done-uri <uri>] [--segmenter <name>] [--skip-validation] [--config path] [--scratch dir] [--max-frames n] (--set a.b.c=v)*` (worker/__main__.py:720-761). --set is dotted-path setattr with bool/int/float coercion (_coerce, 48-58).","ComputeJobSpec: {video_uri, out_uri, segmenter, config_overrides: tuple['k=v',...], skip_validation: bool} (compute/base.py:34-51). ComputeResult.returncode: 0 ok / 2 validation-or-config / 3 segmenter / 4 poll-timeout-or-bad-marker / 5 terminated-without-marker (CLI, worker/__main__.py:174-214).","Forced container overrides on EVERY dispatch: --set deployment.compute_target=local_gpu --set indexing.detector_impl=native (cloud_run.py:57-60) ? appended AFTER spec.config_overrides so they win positionally (applied last by _apply_overrides).","Done-marker: <out_uri>/_dispatch/<uuid4>.done JSON {video_id, returncode, segment_count, status}

(worker/__main__.py:217-241; cloud_run.py:252).","Baked control-plane config.json shape (gen_service_yaml.build_config:62-87): deployment{profile,compute_target}, indexing{skip_ai_handoff:true, default_segmenter:wasb_hybrid}, storage{backend:gcs, output_root, input_backend:gcs, input_root:<bucket>/videos}, security{edge_auth_enabled, cf_team_domain, cf_access_aud, allowed_video_root:/tmp/apphome/output, max_part_mb:24}."],"gotchas":["The worker Job runs with NO DEPLOYMENT_PROFILE, so PROFILE_PRESETS never apply in the container ? every preset-resolved knob the worker must honor has to be explicitly forwarded as --set by the control plane (orchestration.py:598-619) or set as Job env. Forgetting this silently reverts the worker to model defaults (the #506/#507 lesson: decode_backend fell back to cpu/batch-8 until forwarding landed).","#513 incident (2026-07-07): control-plane validated a clip at min_blur_threshold=8.0, then the worker RE-validated with its own default 20.0 and rejected it AFTER the paid GPU dispatch ? fixed by ComputeJobSpec.skip_validation + --skip-validation (worker/__main__.py:273-291). Any plan moving validation ONTO the L4 (#521/#524) must forward the control plane's resolved validation.* config to the worker (the --set precedent), or the same divergence reappears in reverse; it also interacts with the #518 validation-verdict cache, whose fingerprint folds validation.* config + sport + validator set (orchestration.py:814, database get/set_validation_cache).","Version skew is one-directional: a NEW worker CLI flag makes an OLD worker image argparse-abort (rc 2) BEFORE writing the done-marker; pre-#515 that meant a ~65-min opaque poll timeout, now RemoteExecutionFailed fails fast (#515, cloud_run.py:320-356). Rule: roll the worker image FIRST, control plane second (deploy/cloudrun/README.md:79-90). A new phase-placement dispatch flag inherits this constraint.","_apply_overrides does setattr on NESTED config models which do NOT have validate_assignment (model_config with validate_assignment=True exists only on AppConfig, models.py:554-557) ? so --set values on nested Literal fields (e.g. a future deployment.phase_placement.validation) are NOT validated at assignment; a typo silently becomes a string the consumer must handle. The forced 'deployment.compute_target=local_gpu' override works today because DeploymentConfig is a direct child accessed via getattr chain, not because it's validated.","Unknown NESTED config.json keys are silently dropped by Pydantic default extra='ignore' (only a loader WARNING via _unknown_key_paths, loader.py:82-105, 164-173); top-level unknowns hard-fail (AppConfig extra='forbid'). WasbIndexConfig's docstring (models.py:69-74) records a whole section being silently dropped this way. So a new-schema config.json deployed against an older image loses the new section with only a log warning.","Startup-check failure semantics differ by process: the HTTP server sys.exit(1)s (main.py:1238-1244, include_exposure=True); the worker returns rc 2 (worker/__main__.py:153-171). The cloud-serving profile has TWO fatal checks: STORAGE_INCOHERENT (non-gcs/s3 storage) and AI_HANDOFF_KEY_MISSING (skip_ai_handoff=False without ${AI_PROVIDER}_API_KEY, added after the 2026-07-03 silent-0-rally litmus). A placement knob mismatch (e.g. validation->cloud_run_l4 with compute_target!='cloud_run') belongs here as a new fatal check.","deployment.compute_target='' is meaningful: it is the default AND the no-profile state; load_config re-stamps the canonical target when a profile is active but config.json blanked it (loader.py:200-206), and warns (not errors) on an explicit profile/target mismatch (207-214). A per-phase knob should mirror this '' = legacy-default tri-state so default configs stay byte-identical.","compute_target reaches decisions via defensive getattr chains everywhere ? getattr(getattr(config,'deployment',None),'compute_target','') (orchestration.py:200, job_worker.py:63,360, compute/__init__.py:65) ? never direct attribute access; a new field must tolerate dict-configs/absent sections the same way.","The per-request escape hatch: ProcessRequest.compute 'local'|'cloud' (main.py:269) overrides the server default via get_compute_backend(target_override=...) (compute/__init__.py:48,62-66), guarded by _cloud_available (orchestration.py:190-206) and a weights-present fail-fast for 'local' on a weightless control plane (orchestration.py:480-498). A phase-placement design must decide whether the per-request override also moves validation.","PROPOSED #521 KNOB SHAPE (idiomatic slot-in): (1) models.py ? new `PhasePlacementConfig(BaseModel)` with fields `validation/detect/compile: Literal['', 'control_plane', 'cloud_run_l4'] = ''` ('' = today's behaviour), added to DeploymentConfig as `phase_placement: PhasePlacementConfig = Field(default_factory=PhasePlacementConfig)` right after compute_target (models.py:471) ? nesting precedent: ValidationConfig.relevance, StitchingConfig.watermark, IndexingConfig.wasb. (2) presets.py ? add `PHASE_PLACEMENTS: tuple = ('control_plane','cloud_run_l4')` beside COMPUTE_TARGETS (line 28) and, when flipping, extend PROFILE_PRESETS['cloud-serving']['deployment'] with `'phase_placement': {'validation': 'cloud_run_l4'}`; pin Literal/tuple parity in tests/test_config_deployment.py (mirror lines 77-97). (3) Consume it in orchestration._execute_processing before notify('validating') (~line 802) via the getattr-chain idiom; DEFAULT-OFF ('' behaves exactly as today) per the D-series additive convention documented in DeploymentConfig's own docstring (models.py:456-465). (4) Disarm recursion by extending _DEFAULT_CONTAINER_OVERRIDES in cloud_run.py:57-60 (e.g. force phase_placement empty in the container), and forward validation.* thresholds as --set. Closest single precedent END-TO-END:

deployment.remote_poll_max_wait_sec ? field (models.py:483) -> consumed at the seam (compute/__init__.py:78-83); and for worker-side effect: indexing.wasb.decode_backend ? field (models.py:131) -> preset (presets.py:66-71) -> forwarded --set (orchestration.py:609-619) -> env override at the leaf (native_wasb_runner.py:132) -> rollback lever documented (deploy/cloudrun/README.md:67-78)."],"config_knobs":["deployment.profile = '' (Literal ''|truly-local|cloud-serving|cpu-mock; models.py:468; DEPLOYMENT_PROFILE env wins)","deployment.compute_target = '' (Literal ''|local_gpu|cloud_run|cpu_mock; models.py:471 ? the ONLY where-does-detect-run knob today)","deployment.max_concurrent_remote_jobs = 4 (models.py:477; drain concurrency for cloud_run, #466)","deployment.remote_poll_max_wait_sec = 3900.0 (models.py:483; must outlast the Job's --task-timeout 3600)","indexing.detector_impl = 'auto' (models.py:210; auto=WSL, native=Linux GPU, stub=CPU mock; RALLY_DETECTOR_IMPL env overrides ? trajectory_hybrid.py:93)","indexing.skip_ai_handoff = False (models.py:289; cloud-serving preset False, live baked control-plane config pins true)","indexing.default_segmenter = 'yolo_hybrid' (models.py:203; live control plane pins wasb_hybrid, #479)","indexing.wasb.decode_backend = 'cpu' (models.py:131; cloud-serving preset 'nvdec_resident'; WASB_DECODE_BACKEND env beats all)","indexing.wasb.batch_size / prefetch_batches = None (models.py:118/121; preset 64/4; forwarded as --set only when set)","indexing.wasb.device = 'cuda' (models.py:91; WASB_DEVICE env overrides)","storage.backend = 'local' / storage.input_backend = 'local' / storage.input_root = '' / storage.output_root = '' (models.py:421-433; cloud dispatch REQUIRES output_root, orchestration.py:580-586; local-upload staging requires a cloud input_root, orchestration.py:523-546)","validation.min_blur_threshold = 20.0 (+ min_resolution_*, min_fps, pts_max_gap_seconds, relevance.*; models.py:24-43 ? the thresholds whose control-plane-vs-worker divergence caused #513)","security.edge_auth_enabled = False (models.py:403; gates the exposure startup checks)","billing.enabled = False / flywheel.enabled = False (models.py:499/521; default-off, irrelevant to placement but same additive convention)"]},{"summary":"VALIDATION is a CPU-only, pluggable gate that runs on the control-plane (CP) inside _execute_processing BEFORE segmentation. A global ValidationRegistry holds 6 validators registered in dependency order: format_check (ffprobe container probe, seeds context['ffprobe_meta']), resolution_fps_check, pts_continuity_check, then three frame-decoding FrameSamplingValidators ? scene_cut_check, blur_check, relevance_check ? which since #512 share ONE cv2 decode pass (frame_sampler.py). The verdict is a List[ValidationResult] persisted in three places: the videos.validation_results JSON column, the #518 validation_cache SQLite table (keyed by video_id = MD5 of file bytes + a config fingerprint), and the job result JSON ('results' key) the SPA polls. A FAIL sets videos.status='failed' and returns a status='failed' job result with a joined failure message; segmentation never runs. On the cloud_run dispatch path the CP is the authoritative gate: it validates locally, then sends ComputeJobSpec(skip_validation=True) so the worker's --skip-validation flag (#513) skips its own re-validation (the worker's config is NOT fully forwarded and once rejected a CP-passed clip). Validation needs the WHOLE local file (full-file ffprobe demux + frame samples up to the last frames) but zero GPU ? which is exactly why #524 wants a phase_placement knob to move it onto the L4 worker where the bytes already land.","key_files":[{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/pipeline/validators/base.py","role":"Registry + ABCs + the #518 config fingerprint","key_lines":"23: _VALIDATION_FINGERPRINT_SCHEMA=1 (manual bump for logic changes); 26-30: ValidationResult{validator_name,passed,message,details}; 33-58: VideoValidator ABC (validate(video_path,config,context)); 61-118: FrameSamplingValidator (declares sample_indices + online _init_state/_consume/_finalize; uses shared pass via context or standalone fallback); 121-186: ValidationRegistry ? run_all_with_context (129-154) swallows per-validator exceptions into failed results, _prepare_frame_samples (156-181) runs the #512 shared decode best-effort; 188-228: config_fingerprint = sha256 of {schema, sorted validator names, sport, full validation config dump}; 232: global `registry` singleton"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/pipeline/validators/__init__.py","role":"Validator SET + order (folded into fingerprint)","key_lines":"10-15: register order = format_check, resolution_fps_check, pts_continuity_check, scene_cut_check, blur_check, relevance_check"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/pipeline/validators/frame_sampler.py","role":"#512 shared single-pass decoder for the 3 frame validators","key_lines":"49-50: FRAME_META_KEY/FRAME_STATES_KEY context keys; 56-73: hybrid grab-vs-seek gap threshold (max(2s of frames, 64)); 76-113: run_frame_sampling ? one VideoCapture, union of sample indices, streams frames into per-validator accumulators (1 frame in RAM at a time); 116-143: _decode_and_dispatch monotonic-forward pass (returns early on EOF). Module docstring: 6m47s ? ~8.6s on the 1.4GiB reference clip; frames byte-identical to old per-validator seeks"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/pipeline/validators/format.py","role":"format_check ? ffprobe container/streams; seeds shared context","key_lines":"26-31: hard-fails if path not on local disk; 33-51: ffprobe -show_format -show_streams (timeout via backend/utils/ffmpeg); 61: context['ffprobe_meta']=info

(consumed by resolution_fps + report media block); 74-80: requires 'mp4' in format_name"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/pipeline/validators/pts_continuity.py","role":"pts_continuity_check ? full-file packet demux","key_lines":"25-36: ffprobe -show_entries packet=pts_time,flags over the ENTIRE file (reads every packet header, no decode); 97-112: fails on backward jumps or keyframe gap > config.validation.pts_max_gap_seconds"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/pipeline/validators/scene_cut.py","role":"scene_cut_check ? dense samples, first ~50s only","key_lines":"10-13: cut threshold 0.6, cap 100 samples at 2fps; 25-37: sample_indices; 95-104: fails if cuts/min > validation.max_scene_cuts_per_minute"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/pipeline/validators/blur.py","role":"blur_check ? 15 frames evenly across the WHOLE video","key_lines":"21-24: sample_indices span to end of file; 67-76: avg Laplacian variance < validation.min_blur_threshold ? fail (default 20.0; the #513 incident knob)"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/pipeline/validators/relevance.py","role":"relevance_check ? 10 frames whole-video, sport-keyed court color","key_lines":"22-25: sample_indices; 27-63: _init_state resolves HSV bounds with precedence config.validation.relevance > sport_registry.get(config.sport).get_relevance_config() > hardcoded defaults (why `sport` is in the fingerprint); 111-117: avg green ratio < min_court_ratio ? fail"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/orchestration.py","role":"CP pipeline: hash ? validate (+#518 cache) ? dispatch/segment","key_lines":"69-77: _validation_failure_message joins 'name: message'; 242-258: _cached_process_payload replays stored videos.validation_results; 220-239: submit_time_video_id (gs:// metadata md5 only, None for local/composite); 423-705: _maybe_dispatch_remote ? _failed replays val_results (458), stages local upload to storage.input_root as <root>/<video_id>/<basename> (523-579), re-UPSERTs videos with staged_uri + val_results (577-579), builds ComputeJobSpec(..., skip_validation=True) (626-632), warns-not-fails on worker/CP video_id divergence (647-655); 708+: _execute_processing ? 741-756 'hashing' stage: acquire_local_input + compute_video_id(local_video) (748-751 = the #524(b) redundant re-hash), 781-798 whole-pipeline cache, 802-856 validation: fingerprint (814), cache read+fingerprint match+replay (819-843, val_context stays {}), real run registry.run_all_with_context(local_video, config) (846), best-effort set_validation_cache (851-856); 858-873 FAIL branch: add_video(..., 'failed', results) + status='failed' response; 877-879 PASS: add_video 'processed' BEFORE segmentation"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/infrastructure/database.py","role":"Schema: videos column + #518 v10 validation_cache table","key_lines":"196-204: videos(id TEXT PK, filepath, processed_at, status, validation_results TEXT); 471-496: _migrate_to_v10 CREATE TABLE validation_cache(video_id TEXT PRIMARY KEY, fingerprint TEXT NOT NULL, passed INTEGER NOT NULL, results TEXT NOT NULL, created_at TIMESTAMP) ? standalone, NO FK to videos (writable before the videos UPSERT), one latest verdict per video"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/infrastructure/database_media.py","role":"CRUD for verdict persistence","key_lines":"17-40: add_video UPSERT (never REPLACE ? a 'failed' re-validation must not cascade-delete prior segments); 42-68: set_validation_cache UPSERT; 70-91: get_validation_cache ? {fingerprint, passed, results}; 360-368: clear_video_data explicitly DELETEs the validation_cache row (no FK cascade)"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/worker/__main__.py","role":"L4 worker: pull ? hash ? (validate|skip) ? segment ? push","key_lines":"244-256: cmd_infer loads ITS OWN config.json + --set overrides (validation config NOT forwarded by the CP today); 265-271: storage.fetch to scratch then compute_video_id re-hash; 273-299: validation ? skipped when --skip-validation (282-291) else validator_registry.run_all_with_context; 293-299: results go into a THROWAWAY scratch DB (never reaches the CP DB); 301-338: _fail(rc=2 for validation) pushes status='failed' report.json + done-marker; 91-114: _write_report_json shape {video_id,status,segment_count,video_uri,segments[{start,end}]} ? NO validation results today; 217-241: done-marker {video_id,returncode,segment_count,status}; 753-760: --skip-validation CLI arg (direct CLI still validates by default)"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/compute/base.py","role":"Dispatch contract","key_lines":"34-51: ComputeJobSpec(video_uri, out_uri, segmenter, config_overrides tuple of raw '--set k=v', skip_validation=False + full rationale comment); 54-68: ComputeResult(returncode 0/2=validation-or-config/3=segmenter/4=timeout, outputs_uri, video_id from done-marker, execution)"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/compute/cloud_run.py","role":"Spec ? container argv","key_lines":"250-269: builds `python -m backend.worker infer --video-uri ? --out-uri ? --done-uri ?`; 264-267: appends --skip-validation when spec.skip_validation; 268-269: --set for

each override"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/api/uploads.py","role":"#480 streamed upload identity (the hash #524(b) wants to trust)","key_lines":"185+254: incremental hashlib.md5() updated per part; 299-319: bucket-staging finalize returns {path: stage_uri, video_id: st['hasher'].hexdigest()} ? same bytes/algorithm as compute_video_id, no re-read; 320-335: local finalize computes compute_video_id(final_path) and returns it. NOTE: ProcessRequest (backend/main.py:258-272) has NO video_id field, so the returned id is currently discarded by /api/process ? the drain re-hashes"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/api/jobs.py","role":"Submit-time cache probe + job claim","key_lines":"86-113: submit flow ? submit_time_video_id (gs metadata md5, empty for local uploads) ? probe_process_cache ? claim_process_job(video_id=?) ? cached_process_result instant completion (no drain)"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/api/job_worker.py","role":"How a rejected video surfaces","key_lines":"146-190: run_one_job ? job.status is EXECUTION state ('done' even for a validation FAIL); the pipeline verdict lives in result['status']='failed' + result['message'] ('Video failed validation: ?') + result['results'] (the ValidationResult list); 177-179: set_job_video_id + mark_job_done. SPA polls the job and renders result.status/message"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/config/models.py","role":"Validation config (every knob in the fingerprint)","key_lines":"18-21: ValidationRelevanceConfig(court_green_lower=[35,40,40], court_green_upper=[85,255,255], court_pixels_min_ratio=0.05); 24-37: ValidationConfig(min_resolution_width=1280, min_resolution_height=720, min_fps=29.9, min_blur_threshold=20.0, max_scene_cuts_per_minute=0.5, pts_max_gap_seconds=10.0); 471: deployment.compute_target Literal['','local_gpu','cloud_run','cpu_mock']; 423/433: storage.output_root / input_root"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/reporting/harvest.py","role":"Verdict's third consumer: the observability report","key_lines":"33-44: video_meta_from_context reads val_context['ffprobe_meta'] (cached-verdict replay leaves it {} ? media block degrades to size-only, deliberate); 83-89: _validation_stage records total/passed/failures"}],"flow":["1. Upload: SPA chunked upload ? POST /api/upload/{id}/complete returns {path, video_id} ? video_id is the STREAMED md5 (uploads.py:314-319 for bucket staging via st['hasher'], :329 via compute_video_id for local disk). This id is then DISCARDED: ProcessRequest (main.py:258-272) has no video_id field.","2. Submit: POST /api/process (api/jobs.py:86-113) ? submit_time_video_id(input_ref) resolves a cheap md5 from gs:// object metadata ONLY (orchestration.py:220-239; None for local paths/composite objects) ? probe_process_cache ? on hit, cached_process_result completes the job instantly (replaying videos.validation_results); on miss, job row queued with whatever video_id was known.","3. Drain: job_worker.run_one_job claims the job ? _execute_processing(config, db, req, progress) (orchestration.py:708).","4. 'hashing' stage (orchestration.py:741-756): storage.acquire_local_input pulls/identity-resolves req.video_path to a local file, then compute_video_id(local_video) RE-HASHes the whole file (748-751) ? the redundant read #524(b) targets.","5. Whole-pipeline cache (781-798): existing videos row status='processed' + stored segments + not force ? return _cached_process_payload (validation never re-runs).","6. 'validating' stage (802-856): val_fingerprint = registry.config_fingerprint(config) (814). Unless force: db.get_validation_cache(video_id); if row exists AND fingerprint matches, replay ValidationResult(**r) list ? no decode, val_context={} (819-843). Otherwise registry.run_all_with_context(local_video, config) (846): _prepare_frame_samples does the #512 single shared decode for scene_cut/blur/relevance, format_check seeds ffprobe_meta, each validator returns a ValidationResult (exceptions become failed results). Fresh runs are persisted via db.set_validation_cache(video_id, fingerprint, passed, results) best-effort (851-856).","7. Any failure (858-873): db.add_video(video_id, req.video_path, 'failed', results) UPSERT + return {status:'failed', message:_validation_failure_message(failures), results:[?]}; job_worker marks the job 'done' with that result; SPA shows result.status/message; videos row stays queryable with the failed verdict.","8. PASS (877-879): db.add_video(..., 'processed', results) BEFORE segmentation, then _maybe_dispatch_remote (906): on the cloud path a local upload is staged to <storage.input_root>/<video_id>/<basename> (523-579, re-UPSERTing the videos row to the staged URI with the same val_results), and ComputeJobSpec(..., skip_validation=True) dispatches (626-632) ? cloud_run.py appends --skip-validation to the worker argv (264-267).","9. Worker (backend/worker/__main__.py cmd_infer): fetch to scratch (265) ? compute_video_id re-hash (271, its own second hash of the same bytes) ? validation SKIPPED under --skip-validation (282-291, prints the authoritative-gate notice, val_results=[]) or run with the WORKER's config for direct-CLI runs (289-291) ? validation failure = _fail(2): status='failed' report.json + intervals.csv pushed to <out_uri>/outputs/<video_id>/ and a done-marker {video_id, returncode, segment_count, status} ? the CP's bucket-poll terminates fast.","10. CP ingest (orchestration.py:638-705): non-zero rc ? generic 'cloud job failed (returncode N)' (645-646, no validator detail); success ? _ingest_remote_segments + provenance stamp; the response's 'results' key is always the CP-side val_results."],"contracts":["ValidationResult

(validators/base.py:26-30): {validator_name: str, passed: bool, message: str, details: dict} ? serialized with .model_dump() everywhere; round-trips via ValidationResult(**r) on cache replay (orchestration.py:826-828).","validation_cache table (database.py:487-495, v10): video_id TEXT PK (= MD5 of file bytes, utils/hashing.py::compute_video_id), fingerprint TEXT (sha256 of json {schema:1, validators: sorted 6 names, sport, validation: full ValidationConfig model_dump} ? base.py:219-228), passed INTEGER 0/1, results TEXT (JSON list of ValidationResult dumps), created_at. UPSERT keeps one latest verdict per video (database_media.py:60-68).","videos.validation_results TEXT column (database.py:202): same JSON list; videos.status ? {'stored','processed','failed'} ? 'failed' is the user-visible rejection state.","Job result JSON (job_worker.py:149-151,179): job.status='done' + result={'video_id', 'status':'failed'|'success', 'message', 'results':[ValidationResult dumps], 'play_segments', 'compute':{path,requested,...}} ? the SPA's poll surface.","ComputeJobSpec (compute/base.py:35-51): video_uri, out_uri, segmenter, config_overrides: tuple['k=v',...] (only indexing.skip_ai_handoff + indexing.wasb.* forwarded today ? orchestration.py:598-619; validation.* is NOT forwarded), skip_validation: bool ? worker CLI flag --skip-validation (cloud_run.py:264-267; worker/__main__.py:753-760).","Worker bucket artifacts under <out_uri>/outputs/<video_id>/: report.json {video_id, status:'success'|'failed', segment_count, video_uri, segments:[{start,end}]} (worker/__main__.py:94-114) + intervals.csv; done-marker at a separate --done-uri: {video_id, returncode, segment_count, status} (217-241). NEITHER carries per-validator results/messages/fingerprint today ? the gap a worker-side validation run must fill for the CP to record an identical verdict (CP needs the full ValidationResult list to write videos.validation_results, set_validation_cache, and response['results'], plus?honestly?the validation-config fingerprint the worker actually validated under).","Upload-complete response (uploads.py:314-319, 329-335): {path, video_id, size_bytes, next:'POST /api/process with this path'} ? streamed md5 == compute_video_id digest (same bytes, same algorithm). #524(b) = add video_id to ProcessRequest (main.py:258) and skip the drain re-hash at orchestration.py:748-751.","Worker exit codes: 0 ok, 2 validation-or-config, 3 segmenter Failure, 4 poll-budget timeout, 5 terminated-without-marker (#515) ? compute/base.py:56-58, worker/__main__.py:174-214.","Env/identity: video_id is THE join key for outputs/<md5>/, labels, cache rows; gs:// staged objects expose md5Hash metadata (submit_time_video_id) except composite objects (orchestration.py:223-224)."],"config_knobs":["validation.min_resolution_width = 1280 (config/models.py:25)","validation.min_resolution_height = 720 (models.py:26)","validation.min_fps = 29.9 (models.py:27)","validation.min_blur_threshold = 20.0 (models.py:28) ? the #513 incident knob (CP ran 8.0, worker default 20.0)","validation.max_scene_cuts_per_minute = 0.5 (models.py:29)","validation.pts_max_gap_seconds = 10.0 (models.py:30-37)","validation.relevance.court_green_lower = [35,40,40] / court_green_upper = [85,255,255] / court_pixels_min_ratio = 0.05 (models.py:18-21; sport profile fallback in relevance.py:27-63)","sport (top-level AppConfig field) ? folded into the fingerprint because relevance keys its court profile off it (base.py:203-217)","deployment.compute_target: ''|local_gpu|cloud_run|cpu_mock (models.py:471) ? gates _maybe_dispatch_remote","storage.input_root / storage.output_root (models.py:433/423) ? staging + worker output; both required for the cloud path","NO phase_placement knob exists anywhere yet (repo-wide grep is empty) ? #524(a) introduces it","ffprobe timeout via backend/utils/ffmpeg.py::ffprobe_timeout_sec (used by format/pts/resolution validators)"],"gotchas":["#513 (shipped as skip_validation, orchestration.py:620-632 + worker/__main__.py:273-291): the worker loads its OWN config.json and the CP forwards only indexing.* overrides ? a worker-side re-validation resolved min_blur_threshold=20.0 and REJECTED a 5312x2988 clip the CP had passed at 8.0, AFTER a successful GPU run (2026-07-07). Direct consequence for #524(a): moving validation to the L4 worker requires forwarding the CP's resolved validation.* config + sport (today only raw '--set k=v' config_overrides exist as a channel), or the worker's verdict is computed under a different config than the CP believes.","#518 cache/fingerprint coupling: the fingerprint stored with a verdict is computed by the CP from ITS config (orchestration.py:814); if a worker validates instead, the CP may only honestly write set_validation_cache(video_id, CP_fingerprint, ?) when the worker demonstrably validated under an identical validation config ? otherwise a later config-matched replay serves a verdict that config never produced. Cleanest contract: the worker echoes back the fingerprint it validated under and the CP stores verbatim.","The #518 cache is a pure speedup, never a failure mode: any read/replay fault degrades to a real validation (orchestration.py:815-843); writes are best-effort (851-856); force=true bypasses reuse AND rewrites a fresh verdict; the worker's --skip-validation composes with it so a post-failure reupload reaches segmentation in ~0 validation time.","Cached replay leaves val_context={} ? no ffprobe_meta ? the observability report's media block degrades to size-only (deliberate; orchestration.py:823-825, harvest.py:33-44). A worker-side validation would need to ship ffprobe_meta too if report parity matters.","validation_cache has NO FK to videos (the row is written before the videos UPSERT can

exist); clear_video_data must DELETE it explicitly (database_media.py:360-368) ? any new deletion path must remember this.","Validation needs the WHOLE file, not a prefix: format_check ffprobe needs the container (moov atom can be at EOF for non-faststart MP4s); pts_continuity_check demuxes EVERY packet (full sequential read, no decode ? pts_continuity.py:25-36); blur (15 frames) and relevance (10 frames) sample evenly up to the LAST frames (blur.py:21-24, relevance.py:22-25); only scene_cut is prefix-local (~first 50s, scene_cut.py:25-37). It needs zero GPU (cv2 + ffprobe, CPU-only). So validation placement follows the BYTES ? running it where upload/stitch lands (the L4) eliminates the CP's full-file decode, which is #524(c)'s premise.","_VALIDATION_FINGERPRINT_SCHEMA (base.py:18-23) must be bumped when validator LOGIC changes verdicts for identical bytes+config. Note: #512's byte-identical-frames claim was verified on one box; a different OpenCV/ffmpeg build in the L4 worker image could decode marginally different frames ? worker-side validation that feeds the shared cache should treat this as a logic-change consideration.","Failure surfacing asymmetry: a CP-side validation FAIL yields a rich result (per-validator messages via _validation_failure_message, orchestration.py:69-77) and a 'failed' videos row; a WORKER-side rc=2 currently surfaces to the CP only as 'cloud job failed (returncode 2)' (orchestration.py:645-646) because report.json/done-marker omit validator detail (worker/__main__.py:94-114, 217-241). #524(a) must extend one of those artifacts with the ValidationResult list to keep the UX identical.","The worker's DB is throwaway scratch (worker/__main__.py:293) ? its add_video never reaches the CP DB; ONLY bucket artifacts cross the boundary. Any worker-side verdict must ride report.json/done-marker to be recorded by the CP (db.add_video + set_validation_cache + response['results']).","Re-hash landscape for #524(b): the same bytes are hashed up to THREE times today ? streamed at upload (uploads.py:254), re-hashed by the CP drain (orchestration.py:748-751, after acquire_local_input which may itself re-download a staged URI), and re-hashed by the worker (worker/__main__.py:271, kept as an integrity check ? divergence only WARNS, orchestration.py:647-655). ProcessRequest has no video_id field (main.py:258-272), so the upload's hash is discarded; submit_time_video_id (orchestration.py:220-239) only recovers it for gs:// inputs via metadata (composite objects excluded).","add_video is deliberately UPSERT not REPLACE (database_media.py:24-32): a 'failed' re-validation must not cascade-delete prior good segments ? preserve this if the verdict write moves.","Validator exceptions never crash the run: run_all_with_context converts them to failed ValidationResults (base.py:143-153), and the shared #512 pre-pass swallows its own failures so validators fall back to standalone decode (base.py:171-179). ffprobe timeouts are verdicts, not crashes (format.py:96-101, pts_continuity.py:128-133) ? a slow GCSFuse/bucket mount on the worker could flip a verdict to FAIL; worker-side validation should run on the LOCAL fetched copy, not a fuse mount.","Related shipped incidents for context: #480 streamed-md5 upload staging, #512 shared decode, #513 skip_validation, #515 fail-fast on marker-less termination (rc 5), #518 verdict cache (DB v10)."]},{"summary":"The DOCS+CONVENTIONS subsystem governing any new design work under docs/COMPUTE_DECOUPLED_SERVING/. The main design doc (README.md, ADR-014's project doc) locks 17 decisions (D1?D17) and tracks milestones M0?M12; TESTING_STRATEGY.md defines the 5-layer test/CI conventions; runlogs/ is the durable record of real cloud runs (naming + 8 required sections). PROJECT_DOC_TEMPLATE.md defines the mandatory structure/status-header for any new tracked design doc; DOC_STATUS.md defines lifecycle (DRAFT?IN PROGRESS?DONE?archived) + the §2 archival due-diligence checklist. ADR-012 (Cloud Run+GPU scale-to-zero = THE serving model), ADR-013 (GCP-only compute), and ADR-014 (pure 3-stage core, profile-selected compute+storage, stitch-where-metered) are the ratified constraints that bound the #524 topology options. Critically for #524: the live system today stitches AND validates on the 2-vCPU control-plane (validation-latency runlog §9), which DEVIATES from ratified D4/ADR-014 decision 4 ('in cloud-serving, STITCH runs on the Cloud Run job so it is metered') ? so a phase_placement/L4-primary design largely REALIZES an already-ratified decision rather than needing a new ADR, but it is bounded by D16(a) (Cloud Run JOBS, not Services-with-GPU), D7 (scale-to-zero, no warm pool), D16(b) (libx264 encode for parity; NVENC only behind a deferred config knob), D9 (Cloudflare Access edge auth), and D5 (parity enforced at WINDOW+STITCH byte-level).","key_files":[{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/docs/COMPUTE_DECOUPLED_SERVING/README.md","role":"THE main design doc (ADR-014's project doc); any new serving design must slot into or reference it","key_lines":"3-8: canonical status-header block (Status/Owner/Created/Last-updated · Lifecycle · Tracking-anchors · Relation-to-existing-docs · Naming · Honesty-note with [verified]/[design]/[researched] tags); 12: D14 northstar (Drive round-trip, mobile); 24-44: decisions-locked table D1-D17 (D4 line 31 stitch-on-CloudRun-metered; D7 line 34 scale-to-zero/no-warm-pool; D13 line 40 profile switchable anytime; D15 line 42 cloud spend never silent; D16 line 43 Cloud Run JOBS + bucket-poll, libx264 encode/NVDEC decode-only/NVENC deferred behind knob, <2GB hard cap; D17 line 44 $100/?10k real-verification budget); 74-80: §4.2 profile table (truly-local/cloud-serving/cpu-mock/byo); 82-88: §4.3 config precedence chain (defaults ? profile preset ? config.json ? env ? worker --set) ? where a phase_placement knob must slot; 90-91: §4.4 'Compute placement ? why stitch runs on

Cloud Run' (the section #524 extends); 110-128: §7 M0-M12 progress tracker (SOURCE OF TRUTH; one PR per row, no stacking); 131-144: §8 resolved owner questions (line 140 Q2: stitch co-located, 'job table supports a future CPU-split non-breaking, revisit only on bursty-compile metrics' ? the revisit trigger #524 now has; line 142 Q8 codec; line 143 Q9 input cap); 146-152: §9 DoD; 158-172: changelog convention (one dated entry per landed milestone with PR#, suite count, review workflow id)"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/docs/PROJECT_DOC_TEMPLATE.md","role":"Mandatory skeleton for any new tracked design doc (per CLAUDE.md workflow conventions)","key_lines":"19-25: status header format (> **Status:** DRAFT · **Owner:** · **Created:** · **Last updated:** / Lifecycle / Tracking anchors: '§7 progress tracker is the source of truth; indexed in docs/README.md; pointer in memory current-state + docs/NEXT_STEPS.md' / Relation to existing docs: extends|supersedes|peer-of / Honesty note); 29-33: §0 TL;DR (2-5 sentences + biggest caveat) + §1 problem/goal; 35-40: §2 decisions-locked table (# | Decision | Source/date | Implication); 45-46: §4 design must include a REUSE MAP (new vs reused); 51-52: §6 'Honest limits ? what this does NOT do'; 54-59: §7 tracker legend (?/?/?/?, one small PR per row, branched from origin/master, no stacking); 61-62: §8 open questions split blocking-gates vs nice-to-knows, name who decides; 64-66: §9 DoD checklist ? flip DONE + archive; 68-74: §10 references ([verified] code anchors) + changelog"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/docs/DOC_STATUS.md","role":"Lifecycle + archival register; a new design doc needs a §3a row","key_lines":"14-15: lifecycle legend (DRAFT?IN PROGRESS?DONE/DELIVERED?ARCHIVED, plus LIVING/PROPOSAL/REPORT/CONVENTION); 17-26: §2 archival due-diligence checklist (verify-vs-code ? flip status + completion note ? update ALL inbound refs ? keep ADR lineage ? git mv to archives/past_projects/ ? record move) ? required IN THE SAME PR for any 'bit-flip' launch PR per project CLAUDE.md, enforced by link-check CI (python -m tools.check_md_links); 34: the COMPUTE_DECOUPLED_SERVING register row ('IN PROGRESS ? M1-M9 LARGELY SHIPPED #279-#290'); 23: live docs stay at docs/ top level, only terminal-state work archives"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/docs/archives/decisions/DECISIONS.md","role":"ADR log ? the binding decisions for topology","key_lines":"598-659: ADR-012 (RATIFIED 2026-06-20): GPU-native L4/T4, TPU out; 'Cloud Run + GPU (L4), scale-to-zero, is the supported SERVING deployment model' (608-611) for pay-per-request GPU-seconds = exact per-video bill, zero idle, co-located with the bucket; NVDEC named as the decode lever (612-613); edge auth hard prerequisite before public serving (646-647); GPU stockout reality ? scale-to-zero/multi-region fit (648-649); 7-point seed scope (652-659). 661-684: ADR-013 (RATIFIED 2026-06-20): DO dropped, 'GCP is the sole compute stack for BOTH training and serving' (667); standing default for ALL future compute = GCP; do NOT re-evaluate alternatives without a new ADR (668); DO Spaces allowed as *storage* target only (682). 686-714: ADR-014 (RATIFIED 2026-06-21, refines 011/012, does NOT supersede them): decision 1 (695) pure 3-stage core NEVER branched on profile; decision 2 (696) ComputeBackend=WHERE / StorageBackend=WHAT-bytes, ONE startup config; decision 4 (698) 'In cloud-serving, STITCH runs on Cloud Run (co-located with detect) so its CPU/wall-time + egress are METERED? truly-local stitch intentionally UNmetered'; decision 5 (699) parity at WINDOW+STITCH byte-exact, DETECT same-GPU only; decision 6 (700) smallest-safe-first, one PR per milestone, DEFAULT-OFF, Launch Report on default flips; 714: owner gates D7-D15 recap"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/docs/COMPUTE_DECOUPLED_SERVING/runlogs/README.md","role":"Runlog convention ? every real run / default flip writes a report here","key_lines":"13-14: naming 'DEPLOY_REPORT_<profile>_<YYYY-MM-DD>.md' (descriptive infix used in practice, e.g. _validation-latency_); 16-26: 8 required sections (TL;DR per-stage ?/?/?, resources, exact copy-pasteable commands, run results incl. profile-parity numbers + the metering trio gpu_active_seconds/wall_seconds/bytes_out, reel smoke-test 'ffmpeg -f null -' + 5-frame contact sheet, gotchas, cost reconciliation, teardown $0 audit ? DELETE never stop); 32-38: append-to-index table requirement"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_validation-latency_2026-07-07.md","role":"The single most load-bearing prior-art doc for #524 ? measured phase-by-phase placement + cost data + files the parent issues","key_lines":"55-68: §4a Video 1 before/after table (CP validation 411-412 s pre-#512 ? 183 s live; worker re-val 169.3 s ? 0 s SKIPPED via #513; counterfactual worker re-val on L4 = 69.7 s = 2.43×); 74-82: §4b off-box A/B 1.95×/3.10×/3.23× across codecs, zero decision drift; 113-127: §9 full live-CUJ phase table (upload ~2m55s client ? stage 30 s ? CP validation 183 s on 2 vCPU ? dispatch+cold-start ~55 s ? worker skip-val 4 s ? WASB detect 161.6 s on L4 ? ingest 4 s ? COMPILE/STITCH ~12m55s on the 2-vCPU CP, ~67 s/clip 4K re-encode); 135: '? Finding ? stitch is the scaling bottleneck? serialized by _LOCAL_COMPUTE_LANE, one run pins the control-plane for ~16 min? Filed: #521 (move compile/stitch to the L4 worker + make phase?machine placement config-driven) and #520 (step-level debug tracing)' ? the direct ancestry of #524; 137: per-run cost breakdown (L4 exec ~222 s ? $0.15-0.55 dominant + CP CPU ~960 s @2vCPU ? $0.05 + reel egress 581 MB ? $0.07 + storage

<$0.02 + Gemini $0 ? ?$0.3-0.7 total; 'the stitch's real cost is CONCURRENCY, not dollars')"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_2026-07-04.md","role":"M7 first real L4 run + GPU optimization sweep ? bounds what an L4-primary design may/may not tune","key_lines":"3: 'D17, $300 envelope' (? differs from README D17's $100/?10k ? confirm current cap with owner); 16-25: L4 is CPU-FEED-bound (<1% GPU util), batch64+prefetch4 = 1.44× shipped; 33-35: '? DON'T ? bigger GPU' and 'FP16/TF32/cudnn.benchmark/NVDEC = OPT-IN ONLY, never default (breaks D5/D16 byte-parity)'; 37-49: the remaining ~5× is a parallel-feed change; 52-55: cost ledger (builds ~$0.16 each, runs ~$2-3 L4 time, idle $0)"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_cp-rebuild_2026-07-05.md","role":"Evidence base for #524(b) drop-the-re-hash: streamed-upload md5 == video_id proven live, plus its one boundary","key_lines":"10-13: what shipped (P2 #490 GCS-streamed upload staging; P4 #492 submit-time MD5 + cache-reuse); 40: live proof 'server-side object md5Hash (hex) == the streamed video_id EXACTLY, component_count empty' ? browser uploads stay md5-addressable; 49-63: §5 findings ? composite GCS objects expose NO md5Hash ? submit_time_video_id None ? cache bypass, and the drain path re-hashes on an empty DB (the residual gap; one wasted L4 re-detection ? ~US$0.4)"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_gpu-resident-nvdec_2026-07-07.md","role":"nvdec_resident rollout record ? precedent for env-var-levered, reversible worker capability changes on the L4 image","key_lines":"13-14: nvdec ~1.4× WASB speedup, trajectory reproduced (26479?26481); 34-38: enable/rollback via WASB_DECODE_BACKEND Job env var; 56-57: the dispatch-forwarding gotcha ? the worker Job has NO DEPLOYMENT_PROFILE so presets never applied there; the control-plane must FORWARD resolved indexing.wasb.* knobs on dispatch (fixed in #508/#509; a phase_placement knob faces the same forwarding question)"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/docs/COMPUTE_DECOUPLED_SERVING/TESTING_STRATEGY.md","role":"Companion doc: what any new milestone must prove before merge","key_lines":"31-34: L4 profile-PARITY test = the load-bearing proof (same replay_csv ? byte-identical windows + segment ranges across profiles); 36-39: L5 owner-side real-GPU ? DEPLOY_REPORT in runlogs/; 46-51: CI gates ? per-PR core-touching gate = golden fixtures green on all 3 OSes AND experiment.compare F1-delta ? 0 AND coverage not regressed; 'No real GPU/net/WSL in CI ? ever'; 56-64: runlog contents convention + DoD coupling"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/docs/CONTROL_PLANE_DURABLE_REBUILD.md","role":"The freshest precedent for how a serving design doc is framed as a 'child of COMPUTE_DECOUPLED_SERVING/' while living at docs/ top level (the pattern a #524 doc should copy)","key_lines":"3-17: full status-header including 'Relation to existing docs: ? child of COMPUTE_DECOUPLED_SERVING/ (ADR-014); consolidates issues #471 #480 #482 #483 #484 #485' + honesty-note tags; 21-34: TL;DR style (live incident ? consolidated fix rows P1-P8 + owner-gated P9 deploy)"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/docs/archives/past_projects/DECOUPLED_COMPUTE/05-COST-ATTRIBUTION.md","role":"The original cost-attribution design the M8 ledger implements ? reuse its formula/metering framing rather than reinventing","key_lines":"18-23: cost_usd(job) = ?_resource usage × rate(pricebook_version); metered resources incl. GPU-seconds, egress, Gemini; 27-31: metering-source table (serverless per-invocation report = best; worker-measured gpu_active_seconds vs wall_seconds so bucket-sync isn't billed); 79-80: illustrative ~$0.09/video example (L4 @ $0.00022/s); 91-97: reconciliation + spend guard"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/docs/README.md","role":"Doc index ? a new design doc MUST get an entry (Product/platform plans section)","key_lines":"20: the COMPUTE_DECOUPLED_SERVING index entry (the style to match); 19: the CONTROL_PLANE_DURABLE_REBUILD child-doc entry; 39: PROJECT_DOC_TEMPLATE convention pointer"},{"path":"C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/docs/NEXT_STEPS.md","role":"Live prioritized cross-track list ? gets a pointer once work starts (per template tracking-anchors)","key_lines":"5-8: the 'ACTIVE' banner pattern for the currently-executing serving track; 15-19: P0 prioritized list incl. item 0 (UI-driven alpha) and item 1 (M0-M9 shipped state)"}],"flow":["A new substantial design (like #524's phase_placement / L4-primary topology) starts as a tracked project doc copied from docs/PROJECT_DOC_TEMPLATE.md ? either a companion file inside docs/COMPUTE_DECOUPLED_SERVING/ (peer of TESTING_STRATEGY.md) or, per the freshest precedent (docs/CONTROL_PLANE_DURABLE_REBUILD.md:13), a top-level docs/ file declaring itself 'child of COMPUTE_DECOUPLED_SERVING/ (ADR-014); consolidates issues #520 #521 #524?' in its Relation-to-existing-docs header line","Header block (template lines 19-25): Status DRAFT + owner-gated marker · Owner · Created · Last updated · Lifecycle line · Tracking anchors ('§7 tracker is the source of truth; indexed in docs/README.md; pointer in NEXT_STEPS.md once work starts') · Relation-to-existing-docs · Honesty note; every claim in the body tagged [verified]/[researched]/[design]","Body: §0 TL;DR ? §1 problem (cite the live 2026-07-08 CUJ

finding: validation 183 s + stitch ~13 min both on the 2-vCPU CP, serialized by orchestration._LOCAL_COMPUTE_LANE, ~16 min pinned per run) ? §2 decisions-locked table (inherit D4/D5/D7/D9/D13/D15/D16/D17 verbatim; add new D-rows only for genuinely new owner picks like 'Jobs vs Service-with-GPU for upload-receiving') ? §4 design with an explicit REUSE MAP ? §6 honest limits ? §7 tracker (one default-OFF PR per row, branched from origin/master, no stacking) ? §8 open questions split blocking-gates vs nice-to-knows ? §9 DoD ? §10 references with [verified] code anchors ? changelog","Cross-file registration in the same PR: add an index bullet to docs/README.md (Product/platform plans), a lifecycle row to docs/DOC_STATUS.md §3a, and (once executing) a pointer in docs/NEXT_STEPS.md; the link-check CI job (python -m tools.check_md_links) gates broken links","Milestone execution follows ADR-014 decision 6 / README D6: default-OFF, smallest-safe-first, one PR per milestone row; any PR touching detect/window/stitch CALLERS must pass the TESTING_STRATEGY §3 core-touching gate (golden fixtures green on 3 OSes + experiment.compare F1-delta ? 0 + coverage not regressed); parity is asserted by the L4 profile-parity test pattern (tests/test_profile_parity.py, byte-identical windows + segment ranges)","Real-hardware verification is owner-gated under D17 (README line 44: $100/?10k cap ? but the 2026-07-04 runlog header says '$300 envelope', so re-confirm the current cap): confirm exact command + hourly cost before each spend, smallest viable run, DELETE every resource after; each real run writes DEPLOY_REPORT_cloud-serving_<infix>_<date>.md to runlogs/ with the 8 required sections + the metering trio (gpu_active_seconds/wall_seconds/bytes_out) and appends to the runlogs/README.md index table","Any DEFAULT FLIP (e.g. flipping phase placement to L4-primary) ships a Launch Report record in runlogs/ per [[launch-report-convention]] (README D6, runlogs/README.md:10); a bit-flip PR that drives a tracked project terminal must additionally run the DOC_STATUS §2 archival due-diligence in the SAME PR (project CLAUDE.md rule)","On completion: tick §9 DoD ? flip Status DONE ? run DOC_STATUS §2 (verify-vs-code, completion note, repoint inbound refs, keep ADR lineage) ? git mv to docs/archives/past_projects/ ? record in DOC_STATUS §3e"],"contracts":["Status-header contract (every tracked doc): '> **Status:** ? · **Owner:** ? · **Created:** YYYY-MM-DD · **Last updated:** YYYY-MM-DD' + Lifecycle + Tracking-anchors + Relation-to-existing-docs + Honesty-note lines (PROJECT_DOC_TEMPLATE.md:21-25; live examples COMPUTE_DECOUPLED_SERVING/README.md:3-8, CONTROL_PLANE_DURABLE_REBUILD.md:3-17)","Claim tags: [verified] (checked against code/live logs) / [researched] (external fact) / [design] (proposed) ? required on load-bearing claims","§7 tracker row contract: ID | Deliverable | Depends | Gated? | Statu
Identify CONCRETE gaps: (1) code paths the implementer will hit that no reader covered (e.g. job status API surface, SPA polling contract, DB migration ladder, worker timeout budgets, Cloud Run job args limits), (2) contradictions between readers, (3) missing facts the design doc needs (e.g. existing cost numbers, region/network assumptions, upload presign support in GCS backend). For each gap, go READ the relevant code/doc yourself and FILL it ? return the filled facts with file:line citations, not just questions.

2. assistant (2026-07-08T05:52:34.224Z)

You've reached your Fable 5 limit. Run /usage-credits to continue or switch models with /model.